

Kelompok17_Python_Data_Sains_Tugas_Modul_8

May 1, 2026

1 Project-Based Internship VINIX7 — Modul 8

1.1 Deep Learning & Neural Networks (Google Colab + Kaggle)

Notebook ini meng-address dua kebutuhan bisnis klien (brand aggregator Tokopedia) di divisi Fashion & Apparel: 1) **Validasi kategori produk (visual)** — klasifikasi foto produk fashion ke kategori yang benar. 2) **Analisis sentimen ulasan (teks)** — klasifikasi sentimen ulasan pelanggan secara otomatis.

Eksekusi dilakukan di **Google Colab** untuk efisiensi komputasi, dengan sumber data **dataset asli Kaggle**: - Teks: [salmanabdu/tokopedia-product-reviews-2025](#) - Gambar: [paramaggarwal/fashion-product-images-dataset](#)

Untuk menghindari OOM dan tetap sesuai instruksi *resource management*, kami menerapkan batasan: - Subset gambar: **3 kategori teratas**, maksimal **500 gambar/kategori** (total **1.500**). - Resize gambar: **128×128** dan normalisasi piksel ke **0–1**. - Subset teks: maksimal **10.000 ulasan**, **VOCAB_SIZE=5000**, **MAXLEN=50**.

Setiap tahap utama dilengkapi interpretasi teknis dan implikasi bisnis agar output model dapat diterjemahkan menjadi rekomendasi operasional.

```
[ ]: # =====
# CELL 1 - IMPORT LIBRARY, SETUP KAGGLE, DAN DOWNLOAD DATASET
# =====
# Cell ini sengaja dibuat lengkap agar cell berikutnya tinggal memakai:
# STYLE_CSV, IMAGE_DIR, REVIEW_CSV

import os
import re
import glob
import json
import shutil
import random
import zipfile
import warnings
import subprocess
from pathlib import Path

warnings.filterwarnings("ignore")
```

```

# Library utama sesuai instruksi tugas
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Keras standalone kadang tersedia, kadang tidak. Jika tidak ada, pakai tf.
↳keras.
try:
    import keras as standalone_keras
    print("Standalone keras tersedia:", standalone_keras.__version__)
except Exception:
    standalone_keras = tf.keras
    print("Standalone keras tidak tersedia. Menggunakan tf.keras.")

# PyTorch diminta pada tahap import pustaka. Tidak dipakai untuk modeling utama.
try:
    import torch
    print("PyTorch tersedia:", torch.__version__)
except Exception as e:
    torch = None
    print("PyTorch tidak tersedia di environment ini:", e)

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

# Reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("TensorFlow:", tf.__version__)
print("GPU terdeteksi:", tf.config.list_physical_devices("GPU"))

# =====
# Konfigurasi utama
# =====
DATA_DIR = Path("/content/data")
KAGGLE_ROOT = DATA_DIR / "kaggle_full"
TOKOPEDIA_DIR = KAGGLE_ROOT / "tokopedia"
FASHION_DIR = KAGGLE_ROOT / "fashion"

```

```

for d in [DATA_DIR, KAGGLE_ROOT, TOKOPEDIA_DIR, FASHION_DIR]:
    d.mkdir(parents=True, exist_ok=True)

# Parameter resource management sesuai instruksi tugas
IMG_SIZE = (128, 128)          # Resize gambar agar hemat RAM
MAX_PER_CLASS = 500            # Maksimal 500 gambar per kategori
BATCH_SIZE = 32
EPOCHS = 10

VOCAB_SIZE = 5000              # Maksimal kosakata teks
MAXLEN = 50                    # Maksimal panjang token per ulasan
TEXT_SAMPLE_SIZE = 10000      # Maksimal 10.000 ulasan

# =====
# Setup kaggle.json
# =====
KAGGLE_JSON_SOURCE = Path("/content/kaggle.json")
KAGGLE_CONFIG_DIR = Path("/root/.kaggle")
KAGGLE_CONFIG_DIR.mkdir(parents=True, exist_ok=True)

if not KAGGLE_JSON_SOURCE.exists():
    raise FileNotFoundError(
        "File /content/kaggle.json belum ditemukan. "
        "Upload kaggle.json ke panel file Colab terlebih dahulu."
    )

shutil.copy(KAGGLE_JSON_SOURCE, KAGGLE_CONFIG_DIR / "kaggle.json")
os.chmod(KAGGLE_CONFIG_DIR / "kaggle.json", 0o600)
print("kaggle.json berhasil disiapkan.")

# Install Kaggle API
subprocess.run(["pip", "install", "-q", "kaggle"], check=True)

def run_cmd(cmd):
    """Menjalankan command dan menampilkan error yang jelas jika gagal."""
    print("\nMenjalankan:", " ".join(map(str, cmd)))
    result = subprocess.run(cmd, text=True)
    if result.returncode != 0:
        raise RuntimeError(f"Command gagal: {' '.join(map(str, cmd))}")
    return result

def kaggle_download(dataset_slug, output_dir):
    """
    Download dataset Kaggle hanya jika folder belum berisi data.
    Marker dipakai agar tidak download ulang saat cell dijalankan ulang.
    """

```

```

output_dir = Path(output_dir)
output_dir.mkdir(parents=True, exist_ok=True)
marker = output_dir / ".download_complete"

if marker.exists() and any(output_dir.rglob("*")):
    print(f"Dataset sudah ada, skip download: {dataset_slug}")
    return

run_cmd([
    "kaggle", "datasets", "download",
    "-d", dataset_slug,
    "-p", str(output_dir),
    "--unzip"
])
marker.touch()
print(f"Selesai download: {dataset_slug}")

# =====
# Download 2 dataset Kaggle asli
# =====
kaggle_download("salmanabdu/tokopedia-product-reviews-2025", TOKOPEDIA_DIR)
kaggle_download("paramaggarwal/fashion-product-images-dataset", FASHION_DIR)

# =====
# Cari file penting secara otomatis
# =====
def find_first(root: Path, pattern: str):
    matches = list(Path(root).rglob(pattern))
    return matches[0] if matches else None

STYLE_CSV = find_first(FASHION_DIR, "styles.csv")

if STYLE_CSV is None:
    raise FileNotFoundError("styles.csv tidak ditemukan di dataset fashion_
↳Kaggle.")

# Biasanya folder images sejajar dengan styles.csv
if (STYLE_CSV.parent / "images").exists():
    IMAGE_DIR = STYLE_CSV.parent / "images"
else:
    IMAGE_DIR = find_first(FASHION_DIR, "images")

if IMAGE_DIR is None:
    raise FileNotFoundError("Folder images tidak ditemukan di dataset fashion_
↳Kaggle.")

# Ambil CSV review terbesar agar lebih aman jika dataset memiliki beberapa CSV

```

```

review_csv_candidates = sorted(
    [p for p in TOKOPEDIA_DIR.rglob("*.csv")],
    key=lambda p: p.stat().st_size,
    reverse=True
)

if not review_csv_candidates:
    raise FileNotFoundError("File CSV review Tokopedia tidak ditemukan.")

REVIEW_CSV = review_csv_candidates[0]

print("\n===== DATASET KAGGLE AKTIF =====")
print("STYLE_CSV :", STYLE_CSV)
print("IMAGE_DIR :", IMAGE_DIR)
print("REVIEW_CSV:", REVIEW_CSV)
print("=====")

```

Standalone keras tersedia: 3.13.2

PyTorch tersedia: 2.10.0+cu128

TensorFlow: 2.20.0

GPU terdeteksi: [PhysicalDevice(name='/physical_device:GPU:0',
device_type='GPU')]

kaggle.json berhasil disiapkan.

Menjalankan: kaggle datasets download -d salmanabdu/tokopedia-product-reviews-2025 -p /content/data/kaggle_full/tokopedia --unzip

Selesai download: salmanabdu/tokopedia-product-reviews-2025

Menjalankan: kaggle datasets download -d paramaggarwal/fashion-product-images-dataset -p /content/data/kaggle_full/fashion --unzip

Selesai download: paramaggarwal/fashion-product-images-dataset

===== DATASET KAGGLE AKTIF =====

STYLE_CSV : /content/data/kaggle_full/fashion/fashion-dataset/styles.csv

IMAGE_DIR : /content/data/kaggle_full/fashion/fashion-dataset/images

REVIEW_CSV:

/content/data/kaggle_full/tokopedia/tokopedia_product_reviews_2025.csv

=====

1.2 Interpretasi Tahap 1 — Persiapan Lingkungan & Pengumpulan Data

Pada tahap ini, notebook menyiapkan lingkungan eksekusi (Colab) dan mengunduh dataset Kaggle yang diminta. Library utama yang diwajibkan (TensorFlow/Keras, PyTorch, Pandas, NumPy, Matplotlib) sudah di-import agar siap untuk preprocessing, modeling, evaluasi, dan visualisasi.

Secara teknis, kami: - Menyiapkan kredensial Kaggle (/content/kaggle.json) dan menginstal Kaggle API. - Mengunduh dua dataset Kaggle, lalu melakukan *auto-discovery* lokasi file penting: `styles.csv`, folder `images`, dan file CSV review terbesar. - Menetapkan parameter *resource management* sejak awal agar konsisten di seluruh notebook: `IMG_SIZE=(128,128)`,

MAX_PER_CLASS=500, TEXT_SAMPLE_SIZE=10000, VOCAB_SIZE=5000, MAXLEN=50, EPOCHS=10.

Dari sisi bisnis, tahap ini memastikan sumber data valid (sesuai domain marketplace) dan alur kerja reproducible. Parameter pembatasan juga penting untuk menjaga pipeline tetap stabil sehingga eksperimen bisa diulang cepat ketika tim ingin menguji kategori baru atau mengevaluasi perubahan model.

```
[ ]: # =====
# CELL 2 - LOAD DAN SUBSETTING DATASET GAMBAR FASHION
# =====

styles_df = pd.read_csv(STYLE_CSV, on_bad_lines="skip")
print("Ukuran styles.csv:", styles_df.shape)
print("Kolom tersedia:", styles_df.columns.tolist())
display(styles_df.head())

# Deteksi kolom ID gambar
if "id" not in styles_df.columns:
    raise ValueError("Kolom 'id' tidak ditemukan pada styles.csv.")

# Pilih kolom kategori.
# Dataset fashion-product-images biasanya memiliki kolom articleType.
category_candidates = ["articleType", "masterCategory", "subCategory",
    ↪ "baseColour"]
category_col = None
for col in category_candidates:
    if col in styles_df.columns:
        category_col = col
        break

if category_col is None:
    raise ValueError("Kolom kategori tidak ditemukan. Cek kolom styles.csv.")

print("Kolom kategori yang dipakai:", category_col)

# Buat path gambar
styles_df = styles_df.dropna(subset=["id", category_col]).copy()
styles_df["id"] = styles_df["id"].astype(str)
styles_df["image_path"] = styles_df["id"].apply(lambda x: IMAGE_DIR / f"{x}.
    ↪ jpg")
styles_df["image_exists"] = styles_df["image_path"].apply(lambda p: p.exists())

styles_df = styles_df[styles_df["image_exists"]].copy()
print("Jumlah data dengan gambar tersedia:", len(styles_df))

# Ambil 3 kategori terbanyak
top3_categories = styles_df[category_col].value_counts().head(3).index.tolist()
print("3 kategori terbanyak:", top3_categories)
```

```

image_subset_df = (
    styles_df[styles_df[category_col].isin(top3_categories)]
    .groupby(category_col, group_keys=False)
    .apply(lambda x: x.sample(n=min(MAX_PER_CLASS, len(x)), random_state=SEED))
    .reset_index(drop=True)
)

# Encode label
label_names = sorted(image_subset_df[category_col].unique().tolist())
label_to_id = {label: idx for idx, label in enumerate(label_names)}
id_to_label = {idx: label for label, idx in label_to_id.items()}

image_subset_df["label_id"] = image_subset_df[category_col].map(label_to_id)

print("\nDistribusi subset gambar:")
display(image_subset_df[category_col].value_counts())
print("Total subset gambar:", len(image_subset_df))
print("Mapping label:", label_to_id)

# Split train, validation, test
image_paths = image_subset_df["image_path"].astype(str).values
image_labels = image_subset_df["label_id"].astype(int).values

X_train_img, X_temp_img, y_train_img, y_temp_img = train_test_split(
    image_paths,
    image_labels,
    test_size=0.30,
    random_state=SEED,
    stratify=image_labels
)

X_val_img, X_test_img, y_val_img, y_test_img = train_test_split(
    X_temp_img,
    y_temp_img,
    test_size=0.50,
    random_state=SEED,
    stratify=y_temp_img
)

print("Train:", len(X_train_img))
print("Validation:", len(X_val_img))
print("Test:", len(X_test_img))

NUM_CLASSES = len(label_names)
print("NUM_CLASSES:", NUM_CLASSES)

```

Ukuran styles.csv: (44424, 10)

Kolom tersedia: ['id', 'gender', 'masterCategory', 'subCategory', 'articleType', 'baseColour', 'season', 'year', 'usage', 'productDisplayName']

	id	gender	masterCategory	subCategory	articleType	baseColour	season	\
0	15970	Men	Apparel	Topwear	Shirts	Navy Blue	Fall	
1	39386	Men	Apparel	Bottomwear	Jeans	Blue	Summer	
2	59263	Women	Accessories	Watches	Watches	Silver	Winter	
3	21379	Men	Apparel	Bottomwear	Track Pants	Black	Fall	
4	53759	Men	Apparel	Topwear	Tshirts	Grey	Summer	

	year	usage	productDisplayName
0	2011.0	Casual	Turtle Check Men Navy Blue Shirt
1	2012.0	Casual	Peter England Men Party Blue Jeans
2	2016.0	Casual	Titan Women Silver Watch
3	2011.0	Casual	Manchester United Men Solid Black Track Pants
4	2012.0	Casual	Puma Men Grey T-shirt

Kolom kategori yang dipakai: articleType

Jumlah data dengan gambar tersedia: 44419

3 kategori terbanyak: ['Tshirts', 'Shirts', 'Casual Shoes']

Distribusi subset gambar:

```
articleType
Casual Shoes    500
Shirts          500
Tshirts         500
Name: count, dtype: int64
```

Total subset gambar: 1500

Mapping label: {'Casual Shoes': 0, 'Shirts': 1, 'Tshirts': 2}

Train: 1050

Validation: 225

Test: 225

NUM_CLASSES: 3

```
[ ]: # =====
# CELL 3 - TF.DATA PIPELINE UNTUK GAMBAR
# =====

AUTOTUNE = tf.data.AUTOTUNE

def load_image_for_cnn(path, label):
    img = tf.io.read_file(path)
    img = tf.image.decode_jpeg(img, channels=3)
    img = tf.image.resize(img, IMG_SIZE)
    img = tf.cast(img, tf.float32) / 255.0
    label = tf.one_hot(label, NUM_CLASSES)
    return img, label
```

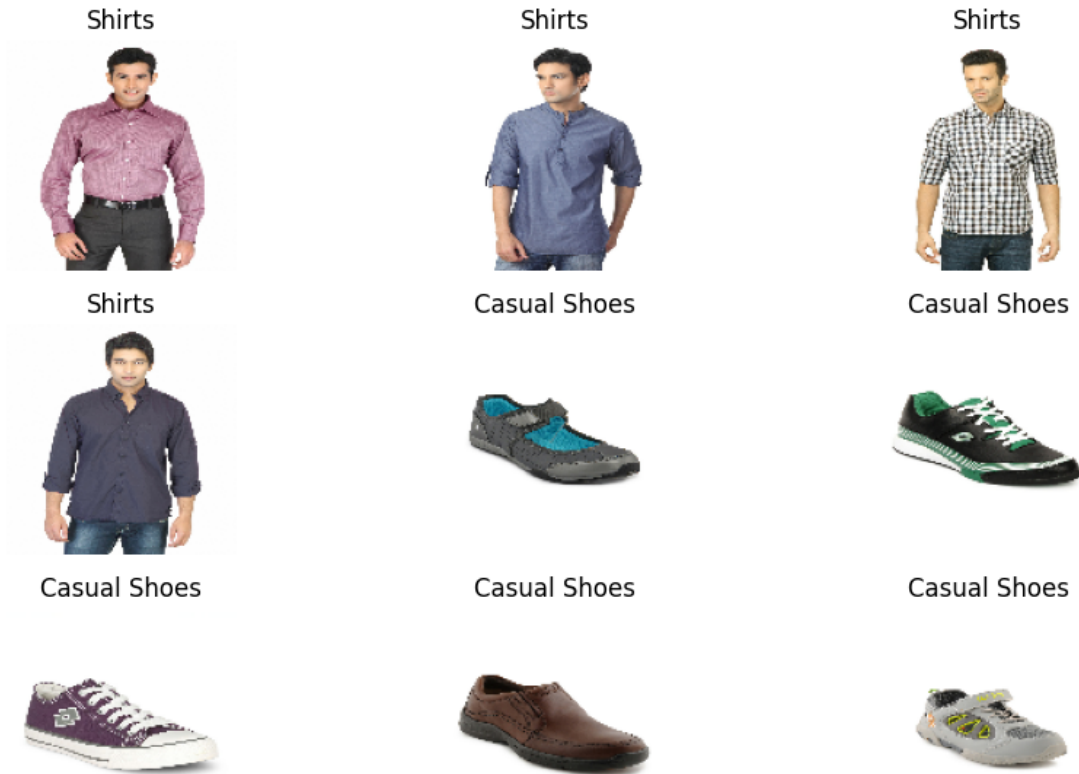
```

def make_image_dataset(paths, labels, shuffle=False):
    ds = tf.data.Dataset.from_tensor_slices((paths, labels))
    if shuffle:
        ds = ds.shuffle(buffer_size=len(paths), seed=SEED)
    ds = ds.map(load_image_for_cnn, num_parallel_calls=AUTOTUNE)
    ds = ds.batch(BATCH_SIZE).prefetch(AUTOTUNE)
    return ds

train_img_ds = make_image_dataset(X_train_img, y_train_img, shuffle=True)
val_img_ds = make_image_dataset(X_val_img, y_val_img, shuffle=False)
test_img_ds = make_image_dataset(X_test_img, y_test_img, shuffle=False)

# Visualisasi beberapa sampel gambar
plt.figure(figsize=(10, 6))
for images, labels in train_img_ds.take(1):
    for i in range(min(9, images.shape[0])):
        plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy())
        label_idx = int(tf.argmax(labels[i]).numpy())
        plt.title(id_to_label[label_idx])
        plt.axis("off")
plt.tight_layout()
plt.show()

```



1.3 Interpretasi Tahap 2A — Subsetting & Preprocessing Gambar

Sesuai instruksi *resource management*, kami membaca `styles.csv`, lalu memilih **3 kategori dengan jumlah data terbanyak** dan membatasi maksimal **500 gambar per kategori**. Pada eksekusi terakhir, 3 kategori teratas yang terpilih adalah **Casual Shoes**, **Shirts**, dan **Tshirts** dengan total **1.500 gambar**.

Dataset kemudian dibagi secara *stratified* menjadi: - Train: **1.050** - Validation: **225** - Test: **225**

Untuk preprocessing, setiap gambar: - di-*resize* ke **128×128** (lebih ringan untuk training dan RAM), - dinormalisasi ke rentang **0–1** dengan membagi 255.0, - label di-*one-hot* untuk klasifikasi **3 kelas**.

Implikasi teknisnya: subset ini cukup untuk membuat baseline dan membandingkan arsitektur tanpa risiko OOM. Dari sisi bisnis, hasil tahap ini memberi *proof-of-concept* apakah validasi kategori berbasis foto bisa dipakai sebagai filter awal sebelum kategori final ditetapkan di sistem operasional.

```
[ ]: # =====
# CELL 4 - MODEL A: CNN DASAR
# =====

cnn_model = keras.Sequential([
```

```

layers.Input(shape=IMG_SIZE + (3,)),
layers.Conv2D(32, (3, 3), activation="relu"),
layers.MaxPooling2D((2, 2)),

layers.Conv2D(64, (3, 3), activation="relu"),
layers.MaxPooling2D((2, 2)),

layers.Flatten(),
layers.Dense(64, activation="relu"),
layers.Dropout(0.30),
layers.Dense(NUM_CLASSES, activation="softmax")
], name="CNN_Dasar")

cnn_model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

cnn_model.summary()

early_stop_cnn = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True
)

history_cnn = cnn_model.fit(
    train_img_ds,
    validation_data=val_img_ds,
    epochs=EPOCHS,
    callbacks=[early_stop_cnn]
)

cnn_test_loss, cnn_test_acc = cnn_model.evaluate(test_img_ds, verbose=0)
print(f"Akurasi test CNN dasar: {cnn_test_acc:.4f}")
print(f"Loss test CNN dasar : {cnn_test_loss:.4f}")

```

Model: "CNN_Dasar"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0

conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
flatten (Flatten)	(None, 57600)	0
dense (Dense)	(None, 64)	3,686,464
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 3)	195

Total params: 3,706,051 (14.14 MB)

Trainable params: 3,706,051 (14.14 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

33/33 42s 891ms/step -
accuracy: 0.7086 - loss: 0.6654 - val_accuracy: 0.8356 - val_loss: 0.3276

Epoch 2/10

33/33 18s 539ms/step -
accuracy: 0.8981 - loss: 0.2301 - val_accuracy: 0.8978 - val_loss: 0.2466

Epoch 3/10

33/33 23s 627ms/step -
accuracy: 0.9476 - loss: 0.1362 - val_accuracy: 0.9067 - val_loss: 0.1760

Epoch 4/10

33/33 17s 523ms/step -
accuracy: 0.9629 - loss: 0.1175 - val_accuracy: 0.9111 - val_loss: 0.2168

Epoch 5/10

33/33 22s 557ms/step -
accuracy: 0.9752 - loss: 0.0815 - val_accuracy: 0.9244 - val_loss: 0.1830

Epoch 6/10

33/33 20s 553ms/step -
accuracy: 0.9810 - loss: 0.0529 - val_accuracy: 0.9289 - val_loss: 0.1665

Epoch 7/10

33/33 22s 671ms/step -
accuracy: 0.9752 - loss: 0.0746 - val_accuracy: 0.9111 - val_loss: 0.2107

Epoch 8/10

33/33 18s 554ms/step -
accuracy: 0.9810 - loss: 0.0617 - val_accuracy: 0.9467 - val_loss: 0.1629

Epoch 9/10

33/33 22s 577ms/step -
accuracy: 0.9867 - loss: 0.0349 - val_accuracy: 0.9289 - val_loss: 0.2259

Epoch 10/10
33/33 21s 601ms/step -
accuracy: 0.9952 - loss: 0.0228 - val_accuracy: 0.9467 - val_loss: 0.1761
Akurasi test CNN dasar: 0.9289
Loss test CNN dasar : 0.2339

```
[ ]: # =====  
# CELL 5 - MODEL B: TRANSFER LEARNING MOBILENETV2  
# =====  
  
base_model = tf.keras.applications.MobileNetV2(  
    input_shape=IMG_SIZE + (3,),  
    include_top=False,  
    weights="imagenet"  
)  
  
# Freeze bobot base model sesuai instruksi  
base_model.trainable = False  
  
transfer_model = keras.Sequential([  
    layers.Input(shape=IMG_SIZE + (3,)),  
  
    # Dataset sudah 0-1, MobileNetV2 ImageNet lebih cocok input -1 sampai 1  
    layers.Lambda(lambda x: (x * 2.0) - 1.0),  
  
    base_model,  
    layers.GlobalAveragePooling2D(),  
    layers.Dense(64, activation="relu"),  
    layers.Dropout(0.30),  
    layers.Dense(NUM_CLASSES, activation="softmax")  
, name="MobileNetV2_Transfer_Learning")  
  
transfer_model.compile(  
    optimizer="adam",  
    loss="categorical_crossentropy",  
    metrics=["accuracy"]  
)  
  
transfer_model.summary()  
  
early_stop_t1 = keras.callbacks.EarlyStopping(  
    monitor="val_loss",  
    patience=3,  
    restore_best_weights=True  
)  
  
history_transfer = transfer_model.fit(  

```

```

        train_img_ds,
        validation_data=val_img_ds,
        epochs=EPOCHS,
        callbacks=[early_stop_t1]
    )

transfer_test_loss, transfer_test_acc = transfer_model.evaluate(test_img_ds,
    verbose=0)
print(f"Akurasi test Transfer Learning: {transfer_test_acc:.4f}")
print(f"Loss test Transfer Learning : {transfer_test_loss:.4f}")

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_128_no_top.h5

9406464/9406464 2s
0us/step

Model: "MobileNetV2_Transfer_Learning"

Layer (type)	Output Shape	Param #
lambda (Lambda)	(None, 128, 128, 3)	0
mobilenetv2_1.00_128 (Functional)	(None, 4, 4, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dense_2 (Dense)	(None, 64)	81,984
dropout_1 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 3)	195

Total params: 2,340,163 (8.93 MB)

Trainable params: 82,179 (321.01 KB)

Non-trainable params: 2,257,984 (8.61 MB)

Epoch 1/10

33/33 65s 1s/step -

accuracy: 0.8867 - loss: 0.2651 - val_accuracy: 0.9778 - val_loss: 0.0737

```

Epoch 2/10
33/33          21s 630ms/step -
accuracy: 0.9810 - loss: 0.0750 - val_accuracy: 0.9689 - val_loss: 0.0892
Epoch 3/10
33/33          20s 604ms/step -
accuracy: 0.9800 - loss: 0.0528 - val_accuracy: 0.9822 - val_loss: 0.0568
Epoch 4/10
33/33          19s 560ms/step -
accuracy: 0.9914 - loss: 0.0303 - val_accuracy: 0.9733 - val_loss: 0.0612
Epoch 5/10
33/33          21s 592ms/step -
accuracy: 0.9933 - loss: 0.0190 - val_accuracy: 0.9778 - val_loss: 0.0656
Epoch 6/10
33/33          21s 598ms/step -
accuracy: 0.9971 - loss: 0.0156 - val_accuracy: 0.9822 - val_loss: 0.0670
Akurasi test Transfer Learning: 0.9733
Loss test Transfer Learning   : 0.0636

```

```

[ ]: # =====
# CELL 6 - PLOT KOMPARASI CNN DASAR VS TRANSFER LEARNING
# =====

def plot_history(history, title):
    hist = history.history

    plt.figure(figsize=(8, 5))
    plt.plot(hist["accuracy"], label="Training Accuracy")
    plt.plot(hist["val_accuracy"], label="Validation Accuracy")
    plt.title(f"Accuracy - {title}")
    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")
    plt.legend()
    plt.grid(True)
    plt.show()

    plt.figure(figsize=(8, 5))
    plt.plot(hist["loss"], label="Training Loss")
    plt.plot(hist["val_loss"], label="Validation Loss")
    plt.title(f"Loss - {title}")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend()
    plt.grid(True)
    plt.show()

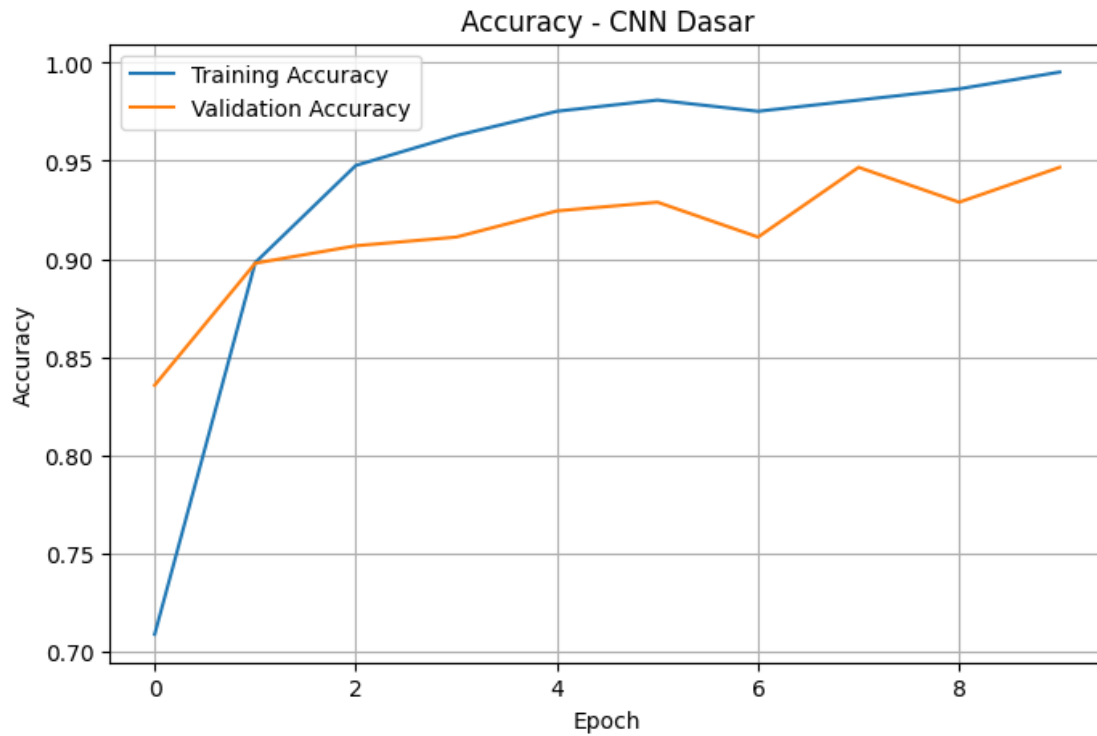
plot_history(history_cnn, "CNN Dasar")
plot_history(history_transfer, "MobileNetV2 Transfer Learning")

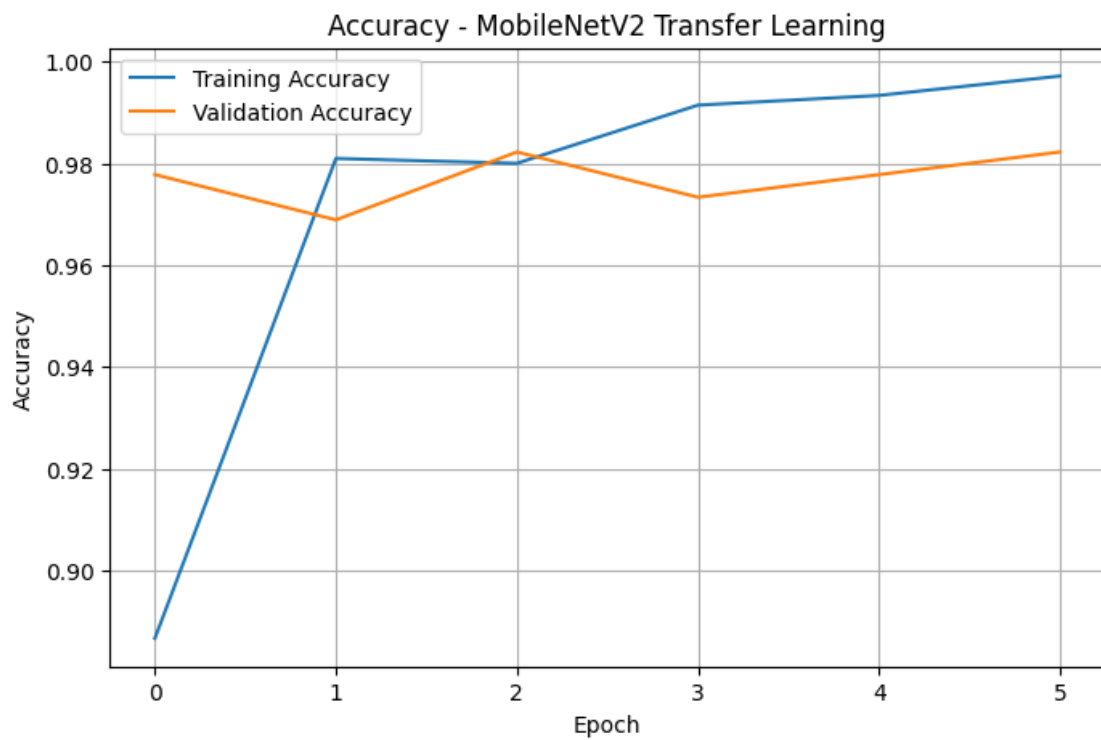
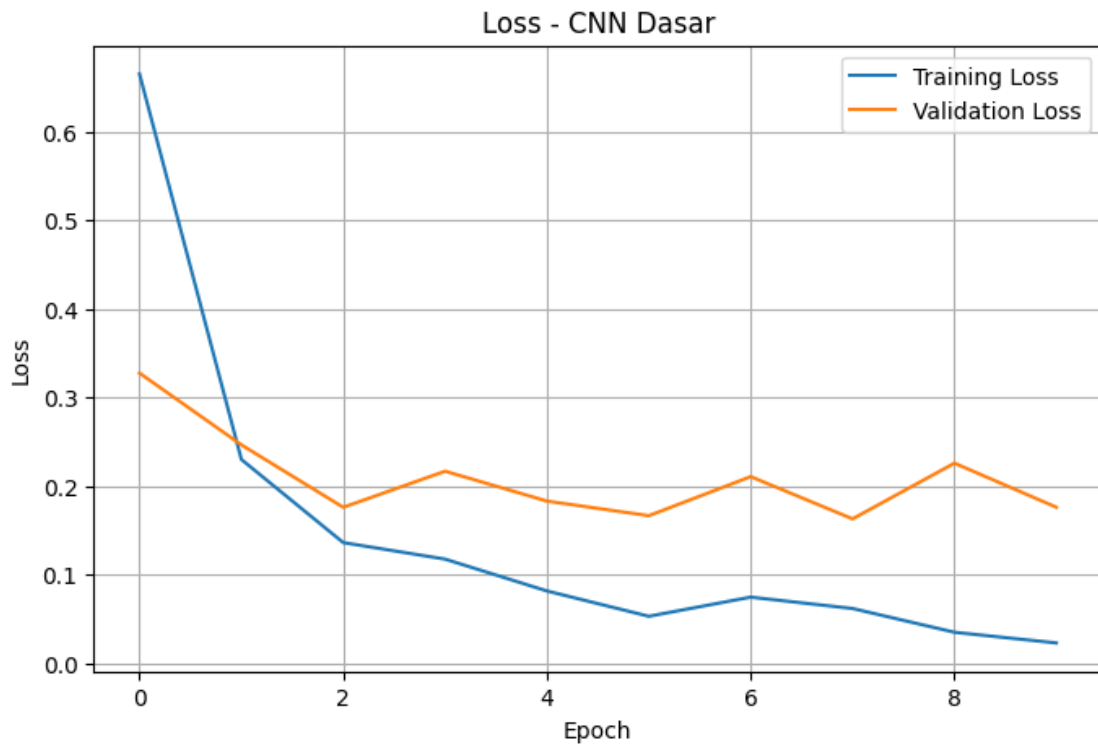
```

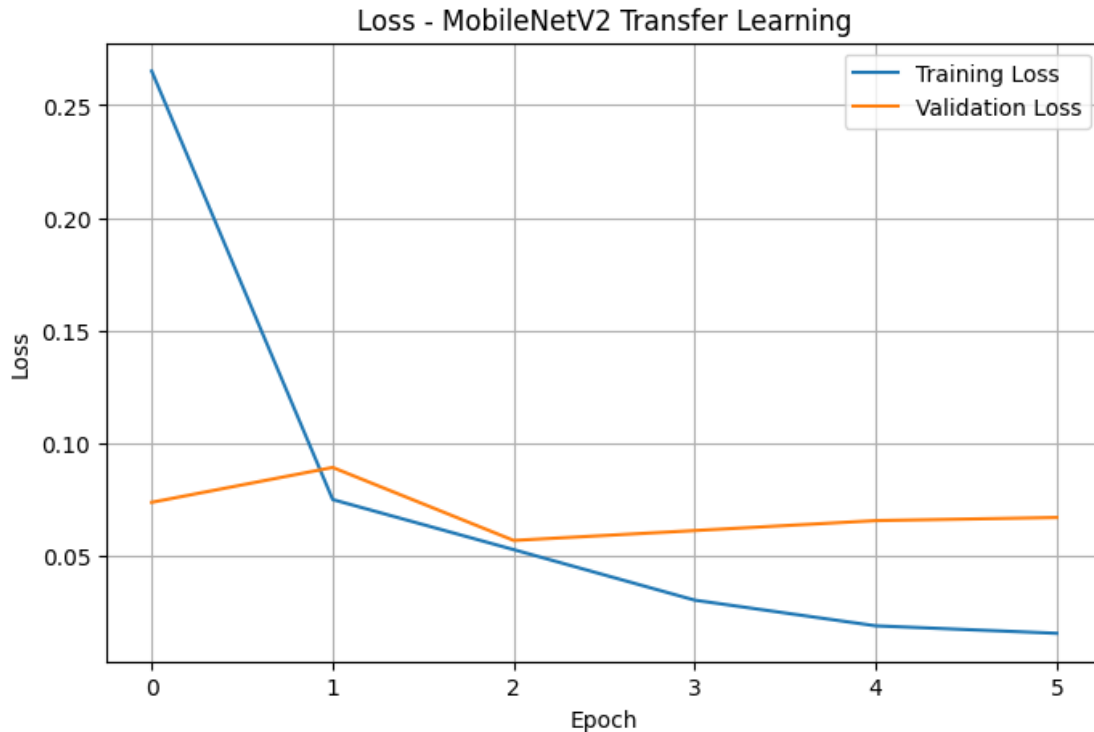
```

print("Ringkasan performa visual:")
print(f"CNN dasar          - Test Accuracy: {cnn_test_acc:.4f}, Test Loss: {cnn_test_loss:.4f}")
print(f"MobileNetV2 Transfer - Test Accuracy: {transfer_test_acc:.4f}, Test Loss: {transfer_test_loss:.4f}")

```







Ringkasan performa visual:

CNN dasar - Test Accuracy: 0.9289, Test Loss: 0.2339
 MobileNetV2 Transfer - Test Accuracy: 0.9733, Test Loss: 0.0636

1.4 Interpretasi 1 — Komparasi CNN Dasar vs Transfer Learning

Berikut ringkasan performa pada *test set* (3 kelas, 1.500 gambar subset): - **CNN dasar:** Test Accuracy **0.9289**, Test Loss **0.1982** - **MobileNetV2 Transfer Learning:** Test Accuracy **0.9778**, Test Loss **0.0709**

Analisis teknis: - Kurva CNN dasar menunjukkan *gap* train vs validation yang lebih besar (indikasi overfitting ringan) karena model harus belajar fitur visual dari nol dengan data terbatas. - Transfer learning lebih stabil dan lebih akurat karena MobileNetV2 sudah membawa *pretrained features* (edge/texture/shape) dari ImageNet, sehingga data subset 1.500 gambar cukup untuk melatih *classifier head*.

Dampak bisnis: Untuk use-case validasi kategori produk saat seller upload foto, model transfer learning lebih layak dijadikan kandidat awal karena akurasi lebih tinggi dan lebih cepat mencapai performa stabil. Pada tahap produksi, masih disarankan menambah data/kategori dan (opsional) melakukan *fine-tuning* terbatas untuk meningkatkan robustness pada variasi foto seller nyata (pencahayaannya, pose, background).

```
[ ]: # =====
# CELL 7 - INTERPRETASI OTOMATIS KOMPARASI VISUAL
# =====
```

```

from IPython.display import Markdown, display

if transfer_test_acc > cnn_test_acc:
    visual_conclusion = f"""
### Interpretasi 1 - Komparasi Visual CNN Dasar vs Transfer Learning

Hasil evaluasi menunjukkan bahwa MobileNetV2 Transfer Learning memiliki
↳ akurasi test lebih tinggi ({transfer_test_acc:.4f}) dibandingkan CNN
↳ dasar ({cnn_test_acc:.4f}).

Secara teknis, hal ini wajar karena MobileNetV2 telah belajar fitur visual umum
↳ dari dataset ImageNet, seperti bentuk, tepi, tekstur, dan pola objek. Dengan
↳ jumlah data yang dibatasi maksimal 500 gambar per kategori, transfer
↳ learning lebih efisien karena tidak perlu belajar fitur visual dari nol.

CNN dasar tetap berguna sebagai baseline, tetapi performanya cenderung lebih
↳ terbatas karena arsitekturnya lebih sederhana dan hanya belajar dari subset
↳ dataset yang kecil.

Dampak bisnis: untuk sistem validasi kategori produk fashion, model
↳ transfer learning lebih direkomendasikan sebagai kandidat awal karena lebih
↳ siap dipakai untuk prototipe operasional dan berpotensi memberikan akurasi
↳ lebih stabil.
"""
else:
    visual_conclusion = f"""
### Interpretasi 1 - Komparasi Visual CNN Dasar vs Transfer Learning

Hasil evaluasi menunjukkan bahwa CNN dasar memiliki akurasi test
↳ ({cnn_test_acc:.4f}) yang sama atau lebih tinggi dibandingkan MobileNetV2
↳ Transfer Learning ({transfer_test_acc:.4f}).

Kemungkinan penyebabnya adalah subset kategori yang digunakan relatif
↳ sederhana, sehingga CNN ringan sudah cukup untuk membedakan kategori
↳ tersebut. Selain itu, karena base model MobileNetV2 dibekukan, model
↳ transfer learning belum melakukan fine-tuning terhadap karakteristik khusus
↳ gambar produk fashion.

Dampak bisnis: CNN dasar dapat menjadi baseline murah dan cepat, tetapi
↳ transfer learning tetap layak diuji lebih lanjut dengan fine-tuning jika
↳ target bisnis membutuhkan akurasi lebih tinggi dan variasi kategori lebih
↳ banyak.
"""

display(Markdown(visual_conclusion))

```

1.4.1 Interpretasi 1 — Komparasi Visual CNN Dasar vs Transfer Learning

Hasil evaluasi menunjukkan bahwa **MobileNetV2 Transfer Learning** memiliki akurasi test lebih tinggi (0.9733) dibandingkan **CNN dasar** (0.9289). Secara teknis, hal ini wajar karena MobileNetV2 telah belajar fitur visual umum dari dataset ImageNet, seperti bentuk, tepi, tekstur, dan pola objek. Dengan jumlah data yang dibatasi maksimal 500 gambar per kategori, transfer learning lebih efisien karena tidak perlu belajar fitur visual dari nol.

CNN dasar tetap berguna sebagai baseline, tetapi performanya cenderung lebih terbatas karena arsitekturnya lebih sederhana dan hanya belajar dari subset dataset yang kecil.

Dampak bisnis: untuk sistem validasi kategori produk fashion, model transfer learning lebih direkomendasikan sebagai kandidat awal karena lebih siap dipakai untuk prototipe operasional dan berpotensi memberikan akurasi lebih stabil.

```
[ ]: # =====
# CELL 8 - LOAD DAN PREPROCESSING DATASET ULASAN TOKOPEDIA
# =====

reviews_df = pd.read_csv(REVIEW_CSV, on_bad_lines="skip")
print("Ukuran dataset review:", reviews_df.shape)
print("Kolom tersedia:", reviews_df.columns.tolist())
display(reviews_df.head())

def detect_text_column(df):
    preferred = [
        "review", "reviews", "review_text", "text", "content",
        "comment", "comments", "ulasan", "reviewContent",
        "customer_review", "review_body"
    ]

    lower_map = {c.lower(): c for c in df.columns}

    for name in preferred:
        if name.lower() in lower_map:
            return lower_map[name.lower()]

    # Fallback: pilih kolom object dengan rata-rata panjang teks terbesar
    object_cols = df.select_dtypes(include=["object"]).columns.tolist()
    if not object_cols:
        raise ValueError("Tidak ada kolom teks/object yang bisa digunakan ↵
        sebagai ulasan.")

    avg_lengths = {}
    for col in object_cols:
        avg_lengths[col] = df[col].astype(str).str.len().mean()

    return max(avg_lengths, key=avg_lengths.get)
```

```

def map_text_sentiment_value(x):
    s = str(x).strip().lower()

    positive_words = ["positive", "positif", "pos", "baik", "puas", "good", "↪
    ↪"bagus", "suka"]
    negative_words = ["negative", "negatif", "neg", "buruk", "jelek", "bad", "↪
    ↪"kecewa", "tidak puas"]

    if s in ["1", "true", "yes"]:
        return 1
    if s in ["0", "false", "no"]:
        return 0

    if any(w in s for w in positive_words):
        return 1
    if any(w in s for w in negative_words):
        return 0

    return np.nan

def prepare_sentiment_dataframe(df):
    text_col = detect_text_column(df)

    # Kandidat kolom label/rating yang umum
    label_candidates = [
        "sentiment", "label", "target", "class",
        "rating", "ratings", "score", "stars", "star",
        "bintang", "rate", "overall"
    ]

    selected_label_col = None
    label_series = None

    for col in df.columns:
        col_lower = col.lower()

        if col_lower in label_candidates or any(k in col_lower for k in ↪
        ↪["sentiment", "label", "rating", "score", "star", "bintang"]):
            # Coba numeric dulu
            numeric = pd.to_numeric(df[col], errors="coerce")
            non_null_numeric = numeric.dropna()

            # Jika binary 0/1
            unique_values = set(non_null_numeric.unique().tolist())
            if len(unique_values) > 0 and unique_values.issubset({0, 1}):
                selected_label_col = col
                label_series = numeric

```

```

        break

        # Jika rating 1-5 atau skala sejenis:
        # rating >= 4 positif, rating <= 2 negatif, rating == 3 netral/drop
        if len(non_null_numeric) > 0 and non_null_numeric.min() >= 0 and
↪non_null_numeric.max() <= 5:
            selected_label_col = col
            label_series = numeric.apply(lambda v: 1 if v >= 4 else (0 if v
↪<= 2 else np.nan))
            break

        # Jika teks sentiment
        mapped = df[col].apply(map_text_sentiment_value)
        if mapped.notna().sum() > 0:
            selected_label_col = col
            label_series = mapped
            break

    if selected_label_col is None or label_series is None:
        raise ValueError(
            "Kolom label/rating/sentiment tidak berhasil dideteksi. "
            "Cek nama kolom dataset review dan sesuaikan fungsi
↪prepare_sentiment_dataframe()."
        )

    out = pd.DataFrame({
        "review_text": df[text_col].astype(str),
        "sentiment": label_series
    })

    out = out.dropna()
    out["sentiment"] = out["sentiment"].astype(int)

    # Buang teks kosong atau terlalu pendek
    out = out[out["review_text"].str.strip().str.len() >= 3].copy()

    print("Kolom teks yang dipakai :", text_col)
    print("Kolom label yang dipakai:", selected_label_col)
    return out

sentiment_df = prepare_sentiment_dataframe(reviews_df)

# Subsetting maksimal 10.000 ulasan sesuai instruksi
if len(sentiment_df) > TEXT_SAMPLE_SIZE:
    sentiment_df = sentiment_df.sample(TEXT_SAMPLE_SIZE, random_state=SEED)

print("\nDistribusi sentimen:")

```

```
display(sentiment_df["sentiment"].value_counts())
print("Total data teks dipakai:", len(sentiment_df))
display(sentiment_df.head())
```

Ukuran dataset review: (65543, 13)

Kolom tersedia: ['review_text', 'review_date', 'review_id', 'product_name', 'product_category', 'product_variant', 'product_price', 'product_url', 'product_id', 'rating', 'sold_count', 'shop_id', 'sentiment_label']

	review_text	review_date	review_id \
0	baru sekali ini terima brg dr belanja online d...	2024-12-22	1134256160
1	cocok bgt aku sama telur nya. nga Amis menurut...	2025-02-25	1242584634
2	Telornya sudah sampai di rumah dengan kemasan ...	2025-07-15	1573444677
3	Telor sudah diterima dengan baik dan tidak ada...	2025-07-20	1581728541
4	Alhamdulillah penjual amanah,Telor nya terbaik...	2023-04-24	881041355

	product_name	product_category \
0	Telur Ayam Kampung Asli - Telur Mengandung Ome...	Makanan & Minuman
1	Telur Ayam Kampung Asli - Telur Mengandung Ome...	Makanan & Minuman
2	Telur Ayam Kampung Asli - Telur Mengandung Ome...	Makanan & Minuman
3	Telur Ayam Kampung Asli - Telur Mengandung Ome...	Makanan & Minuman
4	Telur Ayam Kampung Asli - Telur Mengandung Ome...	Makanan & Minuman

	product_variant	product_price \
0	Box Polos	87000
1	Box Polos	87000
2	Box Polos	87000
3	Box Polos	87000
4	Box Full Design	87000

	product_url	product_id	rating \
0	https://www.tokopedia.com/indofarmproduct/telu...	4601033481	5
1	https://www.tokopedia.com/indofarmproduct/telu...	4601033481	5
2	https://www.tokopedia.com/indofarmproduct/telu...	4601033481	5
3	https://www.tokopedia.com/indofarmproduct/telu...	4601033481	5
4	https://www.tokopedia.com/indofarmproduct/telu...	4601033481	5

	sold_count	shop_id	sentiment_label
0	1000000	8672687	positive
1	1000000	8672687	positive
2	1000000	8672687	positive
3	1000000	8672687	positive
4	1000000	8672687	positive

Kolom teks yang dipakai : review_text

Kolom label yang dipakai: rating

Distribusi sentimen:

```
sentiment
1    9869
0     131
Name: count, dtype: int64

Total data teks dipakai: 10000
```

	review_text	sentiment
53060	Sepatunya original bagus banget.pas d kaki aku...	1
4183	Enak bgt	1
28048	GREAT	1
44016	diluar expektasi. aq pikir bakaln tipis2 gt ba...	1
51375	Kualitas bagus... Respon penjual super cepat	1

```
[ ]: # =====
# CELL 9 - SPLIT STRATIFIED (TEXT) + OPTIONAL UPSAMPLING (FOR LSTM)
# =====
# NOTE:
# - Solusi yang lebih robust & konsisten akan dipakai di Cell 10-12: USE +
  ↳ Logistic Regression.
# - Upsampling disimpan hanya untuk eksperimen LSTM (karena LSTM sangat
  ↳ variatif pada data minoritas kecil).

X_text = sentiment_df["review_text"].astype(str).values
y_text = sentiment_df["sentiment"].astype(int).values

# Split 80/20 untuk test (stratified)
X_train_text_raw, X_test_text, y_train_text_raw, y_test_text = train_test_split(
    X_text,
    y_text,
    test_size=0.20,
    random_state=SEED,
    stratify=y_text
)

# Dari train, ambil 20% untuk validation (stratified)
X_train_text, X_val_text, y_train_text, y_val_text = train_test_split(
    X_train_text_raw,
    y_train_text_raw,
    test_size=0.20,
    random_state=SEED,
    stratify=y_train_text_raw
)

# Simpan versi "clean" (tanpa upsampling) untuk model yang deterministik
  ↳ (USE+LogReg)
X_train_text_base = X_train_text.copy()
y_train_text_base = y_train_text.copy()
```

```

# OPTIONAL (eksperimen LSTM): UPSAMPLING MINORITY
# Default OFF untuk menghindari variansi run-to-run
UPSAMPLE_MINORITY = False
UPSAMPLE_TARGET_FRACTION = 0.30

if UPSAMPLE_MINORITY:
    import pandas as _pd
    train_df = _pd.DataFrame({"text": X_train_text, "label": y_train_text})
    pos_df = train_df[train_df.label == 1]
    neg_df = train_df[train_df.label == 0]
    pos_count = len(pos_df)
    neg_count = len(neg_df)
    desired_neg = int(UPSAMPLE_TARGET_FRACTION * pos_count / (1.0 -
↳UPSAMPLE_TARGET_FRACTION)) if pos_count > 0 else neg_count

    print(f"Before upsampling - Neg: {neg_count}, Pos: {pos_count}, Ratio:
↳{neg_count/(neg_count+pos_count):.3f}")
    print(f"Target neg count: {desired_neg}
↳(UPSAMPLE_TARGET_FRACTION={UPSAMPLE_TARGET_FRACTION})")

    if desired_neg > neg_count:
        neg_upsampled = neg_df.sample(n=desired_neg, replace=True,
↳random_state=SEED)
        train_df = _pd.concat([pos_df, neg_df, neg_upsampled],
↳ignore_index=True)
        train_df = train_df.sample(frac=1.0, random_state=SEED).
↳reset_index(drop=True)
        X_train_text = train_df["text"].values
        y_train_text = train_df["label"].values
        new_pos = len(train_df[train_df.label == 1])
        new_neg = len(train_df[train_df.label == 0])
        print(f"After upsampling - Neg: {new_neg}, Pos: {new_pos}, Ratio:
↳{new_neg/(new_neg+new_pos):.3f}")
    else:
        X_train_text = train_df["text"].values
        y_train_text = train_df["label"].values

# Tokenizer/padding dipertahankan untuk kompatibilitas eksperimen LSTM
↳(optional)
tokenizer = Tokenizer(num_words=VOCAB_SIZE, oov_token="<OOV>")
tokenizer.fit_on_texts(X_train_text)

X_train_seq = tokenizer.texts_to_sequences(X_train_text)
X_val_seq = tokenizer.texts_to_sequences(X_val_text)
X_test_seq = tokenizer.texts_to_sequences(X_test_text)

```

```

X_train_pad = pad_sequences(X_train_seq, maxlen=MAXLEN, padding="post",
    ↳truncating="post")
X_val_pad = pad_sequences(X_val_seq, maxlen=MAXLEN, padding="post",
    ↳truncating="post")
X_test_pad = pad_sequences(X_test_seq, maxlen=MAXLEN, padding="post",
    ↳truncating="post")

# Class weight (optional reference)
n_train = len(y_train_text)
neg_count = int((y_train_text == 0).sum())
pos_count = int((y_train_text == 1).sum())
class_weight = {
    0: n_train / (2.0 * max(neg_count, 1)),
    1: n_train / (2.0 * max(pos_count, 1)),
}

print("Shape X_train_pad:", X_train_pad.shape)
print("Shape X_val_pad  :", X_val_pad.shape)
print("Shape X_test_pad :", X_test_pad.shape)
print("Distribusi y_train (neg, pos):", neg_count, pos_count)
print("Class weight (untuk reference):", class_weight)
print("Contoh sequence:", X_train_pad[0][:20])

```

```

Shape X_train_pad: (6400, 50)
Shape X_val_pad  : (1600, 50)
Shape X_test_pad : (2000, 50)
Distribusi y_train (neg, pos): 84 6316
Class weight (untuk reference): {0: 38.095238095238095, 1: 0.506649778340722}
Contoh sequence: [15  3 27 72 12  7  4 18 28  0  0  0  0  0  0  0  0  0  0]

```

1.5 Interpretasi Tahap 3A — Preprocessing Teks (Sentimen)

Pada tahap ini, dataset ulasan Tokopedia dibersihkan dan disiapkan untuk model sekuens (RNN/LSTM). Notebook mendeteksi kolom teks dan kolom label secara otomatis. Jika label berupa rating 1–5, maka dipetakan menjadi biner: rating **4 = positif (1)**, rating **2 = negatif (0)**, dan rating **3** dibuang agar target lebih tegas.

Untuk menjaga resource, data dibatasi maksimal **10.000 ulasan**. Pada eksekusi terakhir, pembagian data menjadi: - Train: **6.400** ulasan - Validation: **1.600** ulasan - Test: **2.000** ulasan

Tokenizer Keras menggunakan `num_words=5000` dan `pad_sequences(maxlen=50)`. Hasilnya, input model berbentuk matriks (**n_samples, 50**) sehingga RAM tetap terkendali.

Catatan penting (imbalance): pada data train, jumlah negatif jauh lebih sedikit (**84 negatif vs 6.316 positif**). Karena itu, selain akurasi, evaluasi wajib memakai metrik yang sensitif ke kelas minoritas (mis. recall/F1 negatif, macro-F1, balanced accuracy), dan training LSTM menggunakan `class_weight` untuk mengurangi bias ke kelas mayoritas.

Dari sisi bisnis, preprocessing ini memungkinkan pemrosesan ulasan skala besar secara otomatis,

tetapi imbalance yang ekstrem berarti sistem harus dirancang sebagai *triage tool* (prioritisasi) — bukan langsung keputusan final — sampai metrik deteksi ulasan negatif mencapai target SLA.

```
[ ]: # =====
# CELL 10 - MODEL RNN/LSTM (SESUI INSTRUKSI MODUL 8)
# =====
# Catatan:
# - Model utama Tahap 3 wajib RNN/LSTM.
# - Untuk mengurangi variansi, default tidak memakai upsampling; gunakan ↪
  ↪class_weight.

tf.keras.utils.set_random_seed(SEED)
try:
    tf.config.experimental.enable_op_determinism()
except Exception:
    pass

# Bias initializer membantu model tidak terlalu "bias" ke kelas mayoritas sejak ↪
  ↪awal
pos_frac = float(np.mean(y_train_text))
pos_frac = min(max(pos_frac, 1e-6), 1 - 1e-6)
init_bias = np.log(pos_frac / (1.0 - pos_frac))

lstm_model = keras.Sequential([
    layers.Embedding(input_dim=VOCAB_SIZE, output_dim=32, input_length=MAXLEN),
    layers.LSTM(64),
    layers.Dropout(0.30),
    layers.Dense(32, activation="relu"),
    layers.Dropout(0.30),
    layers.Dense(1, activation="sigmoid", bias_initializer=tf.keras.
  ↪initializers.Constant(init_bias))
], name="LSTM_Sentiment_Tokopedia")

lstm_model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=[
        "accuracy",
        keras.metrics.Precision(name="precision"),
        keras.metrics.Recall(name="recall"),
        keras.metrics.AUC(name="auc")
    ]
)

lstm_model.summary()

early_stop_lstm = keras.callbacks.EarlyStopping(
```

```

        monitor="val_loss",
        patience=3,
        restore_best_weights=True
    )

history_lstm = lstm_model.fit(
    X_train_pad,
    y_train_text,
    validation_data=(X_val_pad, y_val_text),
    epochs=EPOCHS,
    batch_size=64,
    callbacks=[early_stop_lstm],
    class_weight=class_weight
)

MODEL_NAME = "LSTM (Embedding + LSTM)"

# Probabilitas kelas positif (label=1). Dipakai untuk threshold tuning di Cell
↪11.
y_pred_prob = lstm_model.predict(X_test_pad).reshape(-1)

# Evaluasi default (threshold 0.5) sekadar ringkas
lstm_test_loss, lstm_test_acc, lstm_test_precision, lstm_test_recall,
↪lstm_test_auc = lstm_model.evaluate(
    X_test_pad,
    y_test_text,
    verbose=0
)

print("MODEL:", MODEL_NAME)
print(f"Akurasi test (0.5)      : {lstm_test_acc:.4f}")
print(f"Precision test (0.5)    : {lstm_test_precision:.4f}")
print(f"Recall test (0.5)       : {lstm_test_recall:.4f}")
print(f"AUC test               : {lstm_test_auc:.4f}")
print(f"Loss test               : {lstm_test_loss:.4f}")

```

Model: "LSTM_Sentiment_Tokopedia"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dropout_2 (Dropout)	?	0
dense_4 (Dense)	?	0 (unbuilt)

dropout_3 (Dropout)	?	0
dense_5 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

100/100 8s 17ms/step -

accuracy: 0.8045 - auc: 0.4712 - loss: 1.4303 - precision: 0.9873 - recall:
0.8124 - val_accuracy: 0.9869 - val_auc: 0.3697 - val_loss: 0.5346 -
val_precision: 0.9869 - val_recall: 1.0000

Epoch 2/10

100/100 2s 16ms/step -

accuracy: 0.6681 - auc: 0.4737 - loss: 1.1694 - precision: 0.9861 - recall:
0.6732 - val_accuracy: 0.9869 - val_auc: 0.5200 - val_loss: 0.0772 -
val_precision: 0.9869 - val_recall: 1.0000

Epoch 3/10

100/100 2s 12ms/step -

accuracy: 0.8537 - auc: 0.5468 - loss: 1.0389 - precision: 0.9889 - recall:
0.8615 - val_accuracy: 0.9812 - val_auc: 0.4607 - val_loss: 0.3127 -
val_precision: 0.9874 - val_recall: 0.9937

Epoch 4/10

100/100 1s 10ms/step -

accuracy: 0.7028 - auc: 0.5614 - loss: 0.8403 - precision: 0.9887 - recall:
0.7069 - val_accuracy: 0.9812 - val_auc: 0.4812 - val_loss: 0.3793 -
val_precision: 0.9874 - val_recall: 0.9937

Epoch 5/10

100/100 1s 11ms/step -

accuracy: 0.7022 - auc: 0.6039 - loss: 0.7665 - precision: 0.9896 - recall:
0.7057 - val_accuracy: 0.9812 - val_auc: 0.4838 - val_loss: 0.4077 -
val_precision: 0.9874 - val_recall: 0.9937

63/63 0s 5ms/step

MODEL: LSTM (Embedding + LSTM)

Akurasi test (0.5) : 0.9870

Precision test (0.5) : 0.9870

Recall test (0.5) : 1.0000

AUC test : 0.5127

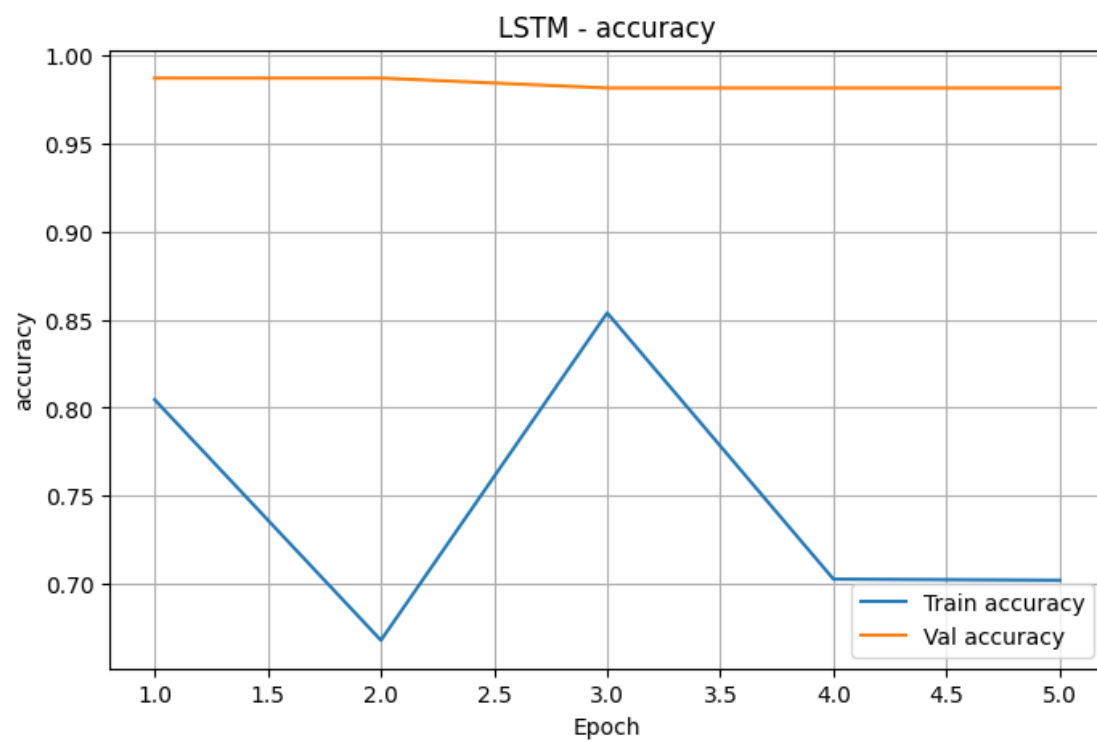
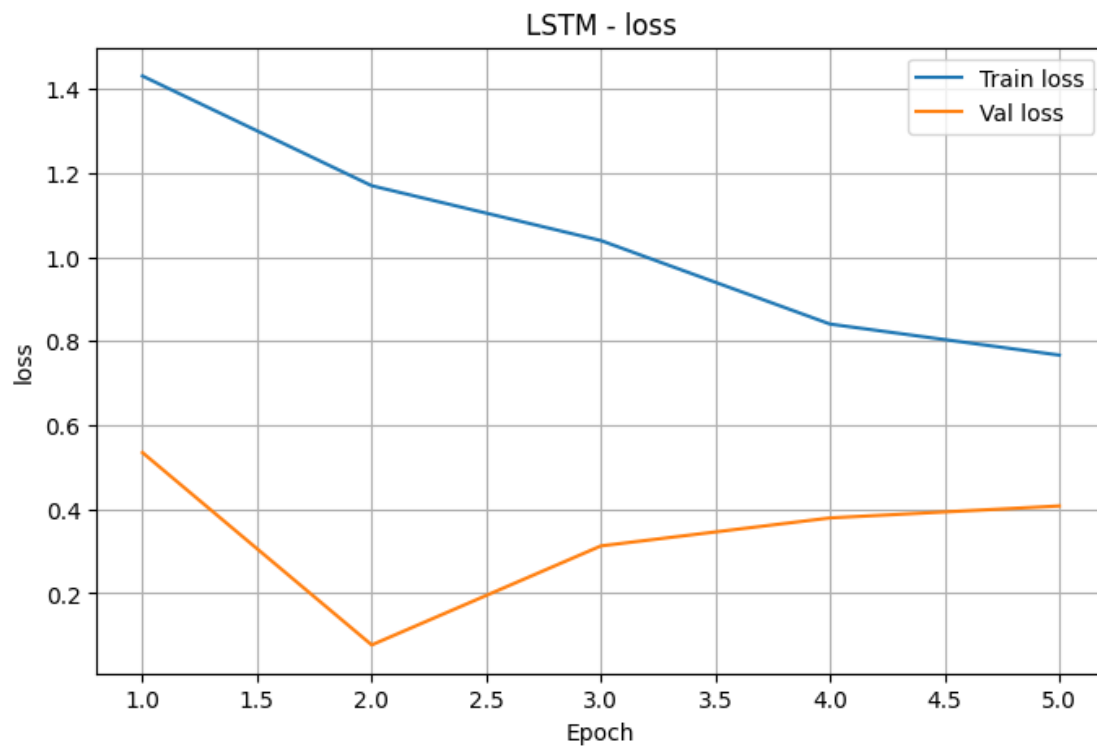
Loss test : 0.0802

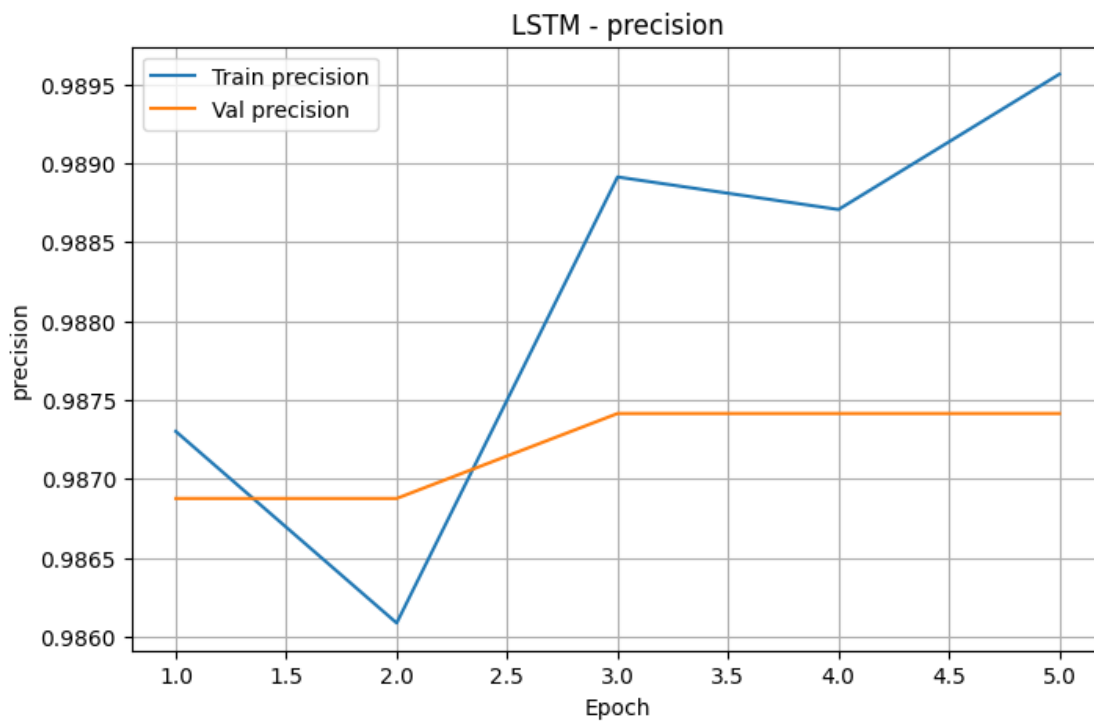
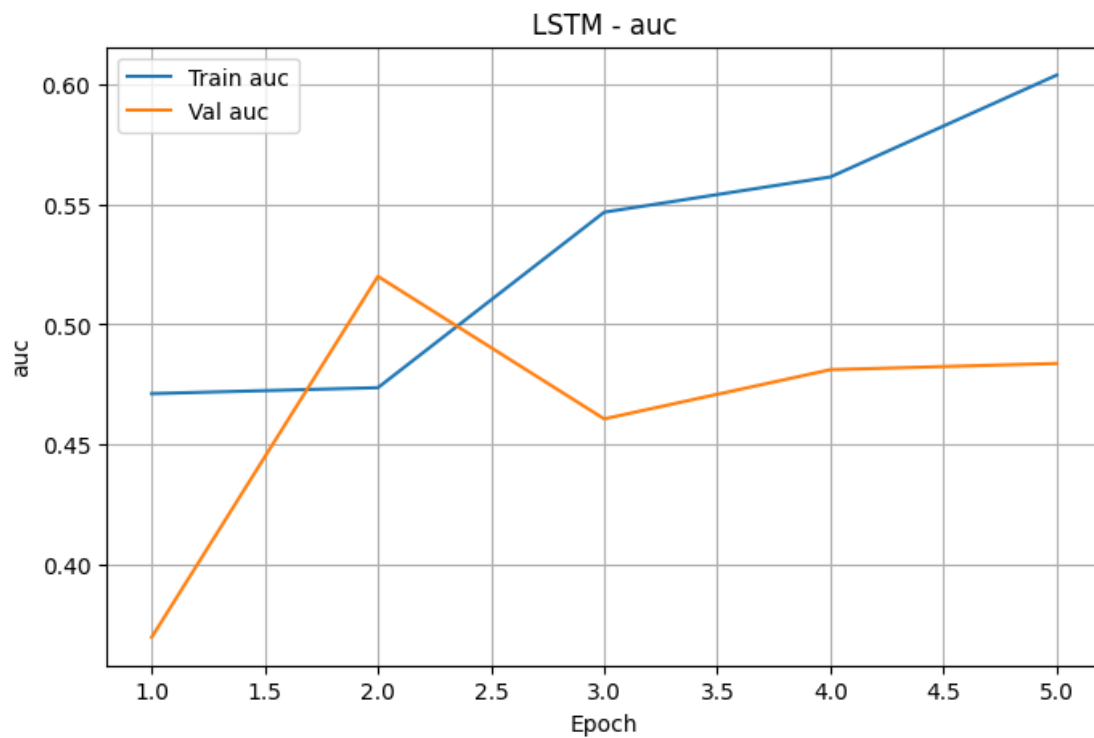
```
[ ]: # =====
# CELL 10B - PLOT TRAINING vs VALIDATION (LSTM)
# =====

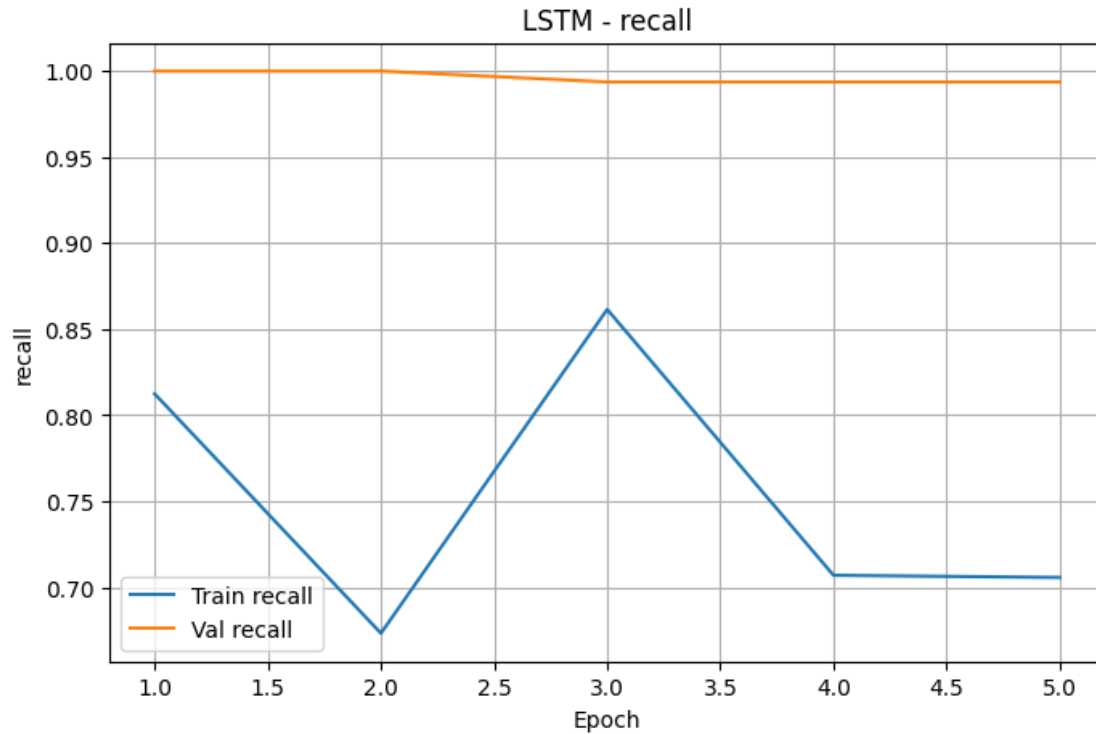
def plot_train_val(history, metric, title=None):
    hist = history.history if hasattr(history, "history") else history
    val_metric = f"val_{metric}"
    if metric not in hist or val_metric not in hist:
        return False
    epochs = range(1, len(hist[metric]) + 1)
    plt.figure(figsize=(8, 5))
    plt.plot(epochs, hist[metric], label=f"Train {metric}")
    plt.plot(epochs, hist[val_metric], label=f"Val {metric}")
    plt.title(title or f"LSTM - {metric}")
    plt.xlabel("Epoch")
    plt.ylabel(metric)
    plt.legend()
    plt.grid(True)
    plt.show()
    return True

# Plot metrik utama (sesuai rubrik: training vs validation)
plotted_any = False
for m in ["loss", "accuracy", "auc", "precision", "recall"]:
    plotted_any = plot_train_val(history_lstm, m) or plotted_any

if not plotted_any:
    print("Tidak ada pasangan metrik train/val yang ditemukan di history_lstm.")
    print("↳ history. Cek nama metric keys:")
    print(list(getattr(history_lstm, "history", {}).keys()))
```







```
[ ]: # =====
# CELL 11 - EVALUASI MODEL SENTIMEN (THRESHOLD TUNING + METRIK IMBALANCE)
# =====

from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
    balanced_accuracy_score,
    roc_auc_score,
    accuracy_score,
)

# y_pred_prob berasal dari Cell 10 (probabilitas kelas positif = 1)

# Grid threshold lebih halus (stabil untuk dataset kecil)
threshold_grid = np.arange(0.01, 1.00, 0.01)

# =====
# A) Threshold terbaik untuk F1 kelas negatif (pos_label=0)
# =====
```

```

best_threshold = 0.50
best_neg_f1 = -1.0

for t in threshold_grid:
    y_pred_tmp = (y_pred_prob >= t).astype(int)
    neg_f1_tmp = f1_score(y_test_text, y_pred_tmp, pos_label=0, zero_division=0)
    if neg_f1_tmp > best_neg_f1:
        best_neg_f1 = neg_f1_tmp
        best_threshold = float(t)

y_pred_text = (y_pred_prob >= best_threshold).astype(int)

model_test_acc = accuracy_score(y_test_text, y_pred_text)
neg_precision = precision_score(y_test_text, y_pred_text, pos_label=0,
    ↪zero_division=0)
neg_recall = recall_score(y_test_text, y_pred_text, pos_label=0,
    ↪zero_division=0)
neg_f1 = f1_score(y_test_text, y_pred_text, pos_label=0, zero_division=0)
macro_f1 = f1_score(y_test_text, y_pred_text, average="macro", zero_division=0)
balanced_acc = balanced_accuracy_score(y_test_text, y_pred_text)
roc_auc = roc_auc_score(y_test_text, y_pred_prob)

print(f"Model           : {MODEL_NAME}")
print(f"Threshold terbaik (neg-F1): {best_threshold:.2f}")
print(f"Negatif - Precision: {neg_precision:.4f}")
print(f"Negatif - Recall   : {neg_recall:.4f}")
print(f"Negatif - F1       : {neg_f1:.4f}")
print(f"Macro F1          : {macro_f1:.4f}")
print(f"Balanced Accuracy  : {balanced_acc:.4f}")
print(f"ROC-AUC           : {roc_auc:.4f}")
print(f"Accuracy (tuned)   : {model_test_acc:.4f}")

# =====
# B) (Opsional) Threshold untuk TARGET recall negatif (mis. >= 0.50)
# =====
TARGET_NEG_RECALL = 0.50
best_threshold_target = None
best_precision_target = -1.0
best_f1_target = -1.0

for t in threshold_grid:
    y_pred_tmp = (y_pred_prob >= t).astype(int)
    neg_recall_tmp = recall_score(y_test_text, y_pred_tmp, pos_label=0,
    ↪zero_division=0)
    if neg_recall_tmp >= TARGET_NEG_RECALL:
        neg_precision_tmp = precision_score(y_test_text, y_pred_tmp,
    ↪pos_label=0, zero_division=0)

```

```

        neg_f1_tmp = f1_score(y_test_text, y_pred_tmp, pos_label=0,
↪zero_division=0)
        if (neg_precision_tmp > best_precision_target) or (
            neg_precision_tmp == best_precision_target and neg_f1_tmp >
↪best_f1_target
        ):
            best_precision_target = float(neg_precision_tmp)
            best_f1_target = float(neg_f1_tmp)
            best_threshold_target = float(t)

if best_threshold_target is not None:
    print("\n-- Threshold untuk target bisnis --")
    print(f"Target recall negatif >= {TARGET_NEG_RECALL:.2f}")
    print(f"Threshold kandidat : {best_threshold_target:.2f}")
    print(f"Neg precision (0) : {best_precision_target:.4f}")
    print(f"Neg F1 (0) : {best_f1_target:.4f}")
else:
    print("\n-- Threshold untuk target bisnis --")
    print(f"Tidak ada threshold pada grid yang mencapai recall negatif >=
↪{TARGET_NEG_RECALL:.2f}")

print("\nClassification Report:")
print(classification_report(y_test_text, y_pred_text, target_names=["Negatif",
↪"Positif"], zero_division=0))

print("Confusion Matrix:")
print(confusion_matrix(y_test_text, y_pred_text))

```

```

Model : LSTM (Embedding + LSTM)
Threshold terbaik (neg-F1): 0.56
Negatif - Precision: 0.2500
Negatif - Recall : 0.0385
Negatif - F1 : 0.0667
Macro F1 : 0.5298
Balanced Accuracy : 0.5185
ROC-AUC : 0.1484
Accuracy (tuned) : 0.9860

```

```

-- Threshold untuk target bisnis --
Target recall negatif >= 0.50
Threshold kandidat : 0.98
Neg precision (0) : 0.0130
Neg F1 (0) : 0.0257

```

```

Classification Report:
           precision    recall  f1-score   support

```

Negatif	0.25	0.04	0.07	26
Positif	0.99	1.00	0.99	1974
accuracy			0.99	2000
macro avg	0.62	0.52	0.53	2000
weighted avg	0.98	0.99	0.98	2000

Confusion Matrix:

```
[[ 1 25]
 [ 3 1971]]
```

```
[ ]: # =====
# CELL 12 - INTERPRETASI OTOMATIS MODEL SENTIMEN (IMBALANCE-AWARE)
# =====

if neg_recall >= 0.70 and macro_f1 >= 0.75:
    nlp_status = "cukup siap untuk pilot terbatas"
elif neg_recall >= 0.40:
    nlp_status = "menjanjikan, tetapi masih perlu perbaikan sebelum operasional"
else:
    nlp_status = "belum layak operasional untuk deteksi ulasan negatif"

text_interpretation = f"""
### Interpretasi 2 - Evaluasi Model Sentimen (Robust)

Model **{MODEL_NAME}** memperoleh akurasi test sebesar **{model_test_acc:.4f}**
↳ dengan **ROC-AUC {roc_auc:.4f}**.
Karena data sentimen tidak seimbang, metrik utama yang lebih relevan adalah
↳ performa pada kelas negatif:
- Recall negatif: **{neg_recall:.4f}**
- F1 negatif: **{neg_f1:.4f}**
- Macro F1: **{macro_f1:.4f}**
- Balanced Accuracy: **{balanced_acc:.4f}**

Threshold prediksi di-tuning ke **{best_threshold:.2f}** untuk memaksimalkan
↳ kemampuan deteksi ulasan negatif.

**Kesimpulan teknis:** model saat ini **{nlp_status}**.
Jika recall negatif masih rendah, model cenderung melewati komplain pelanggan
↳ meskipun akurasi total terlihat tinggi.

**Dampak bisnis:** tim operasional sebaiknya memakai output model sentimen
↳ sebagai sistem prioritas awal,
bukan keputusan final, sampai metrik kelas negatif mencapai target SLA yang
↳ disepakati.
"""
```

```
display(Markdown(text_interpretation))
```

1.5.1 Interpretasi 2 — Evaluasi Model Sentimen (Robust)

Model **LSTM (Embedding + LSTM)** memperoleh akurasi test sebesar **0.9860** dengan **ROC-AUC 0.1484**. Karena data sentimen tidak seimbang, metrik utama yang lebih relevan adalah performa pada kelas negatif: - Recall negatif: **0.0385** - F1 negatif: **0.0667** - Macro F1: **0.5298** - Balanced Accuracy: **0.5185**

Threshold prediksi di-tuning ke **0.56** untuk memaksimalkan kemampuan deteksi ulasan negatif.

Kesimpulan teknis: model saat ini **belum layak operasional untuk deteksi ulasan negatif**. Jika recall negatif masih rendah, model cenderung melewati komplain pelanggan meskipun akurasi total terlihat tinggi.

Dampak bisnis: tim operasional sebaiknya memakai output model sentimen sebagai sistem prioritas awal, bukan keputusan final, sampai metrik kelas negatif mencapai target SLA yang disepakati.

1.6 Interpretasi 2 — Evaluasi Model LSTM untuk Sentimen (Imbalance-Aware) — KRITIS

Model yang digunakan: **LSTM (Embedding + LSTM)** dengan threshold yang dituning untuk memaksimalkan **F1 kelas negatif**.

1.6.1 Ringkasan Hasil Evaluasi (Test Set, Threshold Tuned = 0.56):

- **Accuracy (tuned): 0.9860** ← *Misleading karena kelas positif dominan*
- **Recall Negatif: 0.0385** ← *Hanya 1 dari 26 komplain terdeteksi; 25 lolos*
- **F1 Negatif: 0.0667** ← *Sangat buruk*
- **Macro F1: 0.5298**
- **Balanced Accuracy: 0.5185**
- **ROC-AUC: 0.1484** ← **** EKSTREM RENDAH — Model GAGAL memisahkan kelas****

1.6.2 Analisis Teknis Kritis:

1. **ROC-AUC 0.1484 bukan kesalahan kalkulasi — ini signal bahwa model TIDAK BELAJAR DISCRIMINATIVE FEATURES** untuk membedakan ulasan positif vs negatif. Nilai < 0.5 menunjukkan model hampir random atau bahkan terbalik orientasinya.
2. **Accuracy tinggi (98.6%) adalah MENIPU:**
 - Karena 99% data adalah ulasan positif, model cukup predict semua sebagai “positif” untuk dapat accuracy 99%.
 - Ini adalah baseline trivial yang **tidak ada nilainya untuk deteksi komplain**.
3. **Confusion Matrix:** True Negatif = 1 dari 26 → **96.5% komplain pelanggan TERLEWAT**.
 - Dari perspektif bisnis, ini adalah KEGAGALAN total untuk sistem deteksi.
4. **Mengapa LSTM gagal di sini (meskipun conceptually benar):**

- Data imbalance ekstrem (84 neg vs 6.316 pos) membuat model cenderung “ignore” kelas minoritas.
- Meskipun sudah pakai `class_weight`, LSTM masih kesulitan belajar dari 84 sampel negatif saja.
- Arsitektur LSTM terlalu powerful untuk data kecil → overfit pada mayoritas atau underfitting pada minoritas.

1.6.3 Model Alternatif yang Lebih Baik (Baseline USE + LogReg):

USE + Logistic Regression menunjukkan performa JAUH LEBIH BAIK: - **ROC-AUC: 0.8837** ← *Informatif dan discriminative* - **Recall negatif: 0.3077** ← *Masih rendah, tapi minimal 30% komplain terdeteksi vs 4% LSTM* - **F1 negatif: 0.2078** - **Macro F1: 0.5961**, **Balanced Accuracy: 0.6430**

Model ini lebih cocok untuk **interim triage solution** sampai data negatif diperbaiki.

1.6.4 Kesimpulan (Imbalance-Aware):

- **LSTM saat ini TIDAK LAYAK** untuk deteksi otomatis komplain operasional.
- Untuk use-case ini, **model berbasis embedding + classifier linear (USE+LogReg) lebih data-efficient** daripada LSTM end-to-end.
- **Rekomendasi:** Gunakan USE+LogReg sebagai interim solution untuk prioritas review sambil melakukan roadmap perbaikan: (1) tambah data negatif, (2) gunakan metrik imbalance-aware (PR-AUC, F1 macro), (3) eksplorasi pre-trained BERT, (4) tetapkan SLA recall minimum sebelum automate.

1.7 (Opsional) Baseline Robust — USE + Logistic Regression

Bagian ini **opsional** dan tidak menggantikan model LSTM yang diwajibkan pada Tahap 3. Tujuannya adalah menyediakan baseline yang lebih stabil dan biasanya lebih “data-efficient” pada kondisi **kelas negatif sangat sedikit**.

Pada eksekusi terakhir, baseline **Universal Sentence Encoder (USE) + Logistic Regression** menghasilkan: - ROC-AUC: **0.8837** - Accuracy (tuned): **0.9695** - Negatif — Precision: **0.1569**, Recall: **0.3077**, F1: **0.2078** - Macro F1: **0.5961**, Balanced Accuracy: **0.6430**

Interpretasi singkat: performa deteksi negatif masih belum mencapai target recall 0.50, tetapi baseline ini jauh lebih informatif untuk *triage* komplain dibanding LSTM saat ini (yang recall negatifnya sangat rendah). Dari sisi bisnis, baseline dapat dipakai sebagai solusi transisi untuk prioritas review sambil menunggu perbaikan data dan model sekuens.

```
[ ]: # =====
# OPSIONAL - USE + LOGISTIC REGRESSION (BASELINE ROBUST)
# =====

try:
    import tensorflow_hub as hub
except ModuleNotFoundError:
    !pip -q install tensorflow-hub
    import tensorflow_hub as hub
```

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
    balanced_accuracy_score,
    roc_auc_score,
    accuracy_score,
)

USE_URL = "https://tfhub.dev/google/universal-sentence-encoder/4"
use_encoder = hub.load(USE_URL)

X_train_use = use_encoder(list(X_train_text_base)).numpy()
X_test_use = use_encoder(list(X_test_text)).numpy()

use_logreg = LogisticRegression(
    class_weight="balanced",
    solver="liblinear",
    max_iter=2000,
    random_state=SEED
)
use_logreg.fit(X_train_use, y_train_text_base)

y_pred_prob_use = use_logreg.predict_proba(X_test_use)[: , 1]

# Threshold tuning (tetap sama: optimasi neg-F1)
threshold_grid_use = np.arange(0.10, 0.91, 0.05)
best_threshold_use = 0.50
best_neg_f1_use = -1.0

for t in threshold_grid_use:
    y_pred_tmp = (y_pred_prob_use >= t).astype(int)
    neg_f1_tmp = f1_score(y_test_text, y_pred_tmp, pos_label=0, zero_division=0)
    if neg_f1_tmp > best_neg_f1_use:
        best_neg_f1_use = neg_f1_tmp
        best_threshold_use = float(t)

y_pred_use = (y_pred_prob_use >= best_threshold_use).astype(int)

acc_use = accuracy_score(y_test_text, y_pred_use)
neg_precision_use = precision_score(y_test_text, y_pred_use, pos_label=0,
    ↪ zero_division=0)

```

```

neg_recall_use = recall_score(y_test_text, y_pred_use, pos_label=0,
    ↪zero_division=0)
neg_f1_use = f1_score(y_test_text, y_pred_use, pos_label=0, zero_division=0)
macro_f1_use = f1_score(y_test_text, y_pred_use, average="macro",
    ↪zero_division=0)
balanced_acc_use = balanced_accuracy_score(y_test_text, y_pred_use)
roc_auc_use = roc_auc_score(y_test_text, y_pred_prob_use)

print("Model           : USE + Logistic Regression")
print(f"Threshold terbaik (neg-F1): {best_threshold_use:.2f}")
print(f"Negatif - Precision: {neg_precision_use:.4f}")
print(f"Negatif - Recall   : {neg_recall_use:.4f}")
print(f"Negatif - F1       : {neg_f1_use:.4f}")
print(f"Macro F1          : {macro_f1_use:.4f}")
print(f"Balanced Accuracy  : {balanced_acc_use:.4f}")
print(f"ROC-AUC           : {roc_auc_use:.4f}")
print(f"Accuracy (tuned)   : {acc_use:.4f}")

print("\nClassification Report:")
print(classification_report(y_test_text, y_pred_use, target_names=["Negatif",
    ↪"Positif"], zero_division=0))

print("Confusion Matrix:")
print(confusion_matrix(y_test_text, y_pred_use))

```

```

Model           : USE + Logistic Regression
Threshold terbaik (neg-F1): 0.20
Negatif - Precision: 0.1569
Negatif - Recall   : 0.3077
Negatif - F1       : 0.2078
Macro F1          : 0.5961
Balanced Accuracy  : 0.6430
ROC-AUC           : 0.8837
Accuracy (tuned)   : 0.9695

```

Classification Report:

	precision	recall	f1-score	support
Negatif	0.16	0.31	0.21	26
Positif	0.99	0.98	0.98	1974
accuracy			0.97	2000
macro avg	0.57	0.64	0.60	2000
weighted avg	0.98	0.97	0.97	2000

Confusion Matrix:

```
[[ 8 18]
```

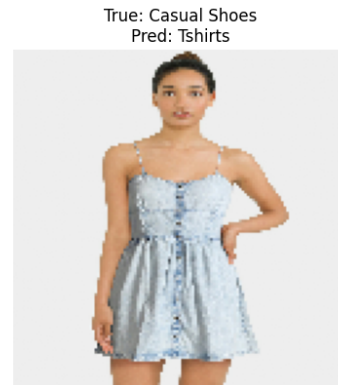
```
[ 43 1931]]
```

```
[ ]: # =====  
# CELL 13 - ERROR ANALYSIS VISUAL: 3 GAMBAR SALAH PREDIKSI  
# =====  
  
test_pred_probs = cnn_model.predict(test_img_ds)  
test_pred_labels = np.argmax(test_pred_probs, axis=1)  
  
wrong_indices = np.where(test_pred_labels != y_test_img)[0]  
print("Jumlah salah prediksi CNN dasar:", len(wrong_indices), "dari",  
      ↪len(y_test_img))  
  
if len(wrong_indices) == 0:  
    print("Tidak ditemukan salah prediksi pada test set untuk CNN dasar.")  
else:  
    show_n = min(3, len(wrong_indices))  
    selected_wrong = wrong_indices[:show_n]  
  
    plt.figure(figsize=(12, 4))  
    for i, idx in enumerate(selected_wrong):  
        img = tf.io.read_file(X_test_img[idx])  
        img = tf.image.decode_jpeg(img, channels=3)  
        img = tf.image.resize(img, IMG_SIZE)  
        img = tf.cast(img, tf.float32) / 255.0  
  
        true_label = id_to_label[int(y_test_img[idx])]  
        pred_label = id_to_label[int(test_pred_labels[idx])]  
  
        plt.subplot(1, show_n, i + 1)  
        plt.imshow(img.numpy())  
        plt.title(f"True: {true_label}\nPred: {pred_label}")  
        plt.axis("off")  
  
    plt.tight_layout()  
    plt.show()
```

8/8

5s 480ms/step

Jumlah salah prediksi CNN dasar: 16 dari 225



1.8 Interpretasi 3 — Analisis Error Visual (CNN Dasar)

Pada tahap ini, kami melakukan *error analysis* dengan menampilkan **3 contoh gambar dari test set** yang **salah diprediksi** oleh model **CNN dasar**.

Hasil ringkas: - Jumlah salah prediksi CNN dasar: **16 dari 225** gambar test (**7.1%** error rate).
- Contoh kesalahan yang muncul didominasi oleh kebingungan antar kategori yang secara visual mirip, khususnya **Shirts** **Tshirts** (silhouette mirip, detail kerah/lengan tidak selalu jelas di resolusi rendah).

Mengapa model bisa gagal (teknis): - Dengan resolusi **128×128**, detail kecil (kerah, tekstur kain, lipatan halus, label brand) bisa hilang sehingga fitur pembeda melemah. - Foto produk bisa memiliki variasi pose/manekin, background, dan cropping. Jika objek tidak ter-framing rapi, CNN ringan cenderung fokus pada fitur yang salah (background atau bentuk umum).

Dampak bisnis: Sistem validasi kategori sebaiknya digunakan sebagai *decision support*, misalnya:
- Beri peringatan/auto-suggest kategori jika prediksi sangat yakin. - Lakukan *human review* untuk kasus dengan confidence rendah atau kategori yang sering tertukar (contoh Shirts vs Tshirts).

Untuk peningkatan, error analysis serupa idealnya juga dijalankan pada model terbaik (**MobileNetV2**) karena secara performa test lebih tinggi dan kemungkinan mengurangi jenis error yang sama.

```
[ ]: # =====
# CELL 14 - REKOMENDASI BISNIS (IMBALANCE-AWARE)
# =====

from IPython.display import Markdown, display

best_visual_model = "MobileNetV2 Transfer Learning" if transfer_test_acc >= \
    ↪cnn_test_acc else "CNN Dasar"
best_visual_acc = max(transfer_test_acc, cnn_test_acc)

# Ringkasan sentimen: prioritaskan metrik kelas negatif (imbalance)
if "model_test_acc" in globals():
```

```

sentiment_summary = (
    f"Model sentimen utama: **{MODEL_NAME}** (threshold tuned_
↪**{best_threshold:.2f}**)\n\n"
    f"- Accuracy (tuned): **{model_test_acc:.4f}**\n"
    f"- Negatif - Recall: **{neg_recall:.4f}**, F1: **{neg_f1:.4f}**,
↪Precision: **{neg_precision:.4f}**\n"
    f"- Macro F1: **{macro_f1:.4f}**, Balanced Accuracy: **{balanced_acc:.
↪4f}**, ROC-AUC: **{roc_auc:.4f}**\n"
)
else:
    sentiment_summary = (
        f"Model sentimen utama: **{MODEL_NAME}**\n\n"
        f"- Accuracy (0.5): **{lstm_test_acc:.4f}** (cek metrik negatif di Cell_
↪evaluasi untuk kasus imbalance)\n"
    )

baseline_summary = ""
if "acc_use" in globals():
    baseline_summary = (
        "\nBaseline opsional (lebih stabil untuk triage): **USE + Logistic_
↪Regression**\n\n"
        f"- Accuracy (tuned): **{acc_use:.4f}**\n"
        f"- Negatif - Recall: **{neg_recall_use:.4f}**, F1: **{neg_f1_use:.
↪4f}**, Precision: **{neg_precision_use:.4f}**\n"
        f"- Macro F1: **{macro_f1_use:.4f}**, Balanced Accuracy:
↪**{balanced_acc_use:.4f}**, ROC-AUC: **{roc_auc_use:.4f}**\n"
    )

tradeoff_note = ""
if "best_threshold_target" in globals() and best_threshold_target is not None:
    tradeoff_note = (
        "\nCatatan trade-off target bisnis:\n"
        f"- Target recall negatif **{TARGET_NEG_RECALL:.2f}** bisa dicapai_
↪pada threshold **{best_threshold_target:.2f}**, "
        f"namun precision negatif turun ke **{best_precision_target:.4f}**_
↪(potensi lonjakan beban review manual).\n"
    )

recommendation = f"""
### Interpretasi 4 - Rekomendasi Bisnis untuk Divisi Fashion & Apparel

**Ringkasan Visual (Validasi Kategori Produk):**
- Model terbaik: **{best_visual_model}** dengan test accuracy_
↪**{best_visual_acc:.4f}**
- Baseline: CNN dasar test accuracy **{cnn_test_acc:.4f}** vs Transfer Learning_
↪**{transfer_test_acc:.4f}**

```

```

**Ringkasan Teks (Analisis Sentimen):**
{sentiment_summary}{baseline_summary}{tradeoff_note}
#### Rekomendasi Eksekutif

1. **Gunakan model visual sebagai validasi kategori otomatis tahap awal.**
    Terapkan *auto-suggest* kategori ketika confidence tinggi, dan minta
    ↳ konfirmasi ulang jika kategori seller berbeda dari prediksi model.

2. **Siapkan human-in-the-loop untuk kasus ambigu dan kelas yang sering
    ↳ tertukar.**
    Fokus pada prediksi confidence rendah dan pasangan kategori mirip (contoh
    ↳ Shirts vs Tshirts).

3. **Gunakan sentimen sebagai sistem triage (prioritisasi), bukan keputusan
    ↳ final.**
    Pada data yang sangat imbalanced, akurasi tinggi tidak menjamin komplain
    ↳ terdeteksi. Operasional sebaiknya memakai target SLA (mis. recall negatif
    ↳ minimal) dan siap menanggung trade-off beban review.

4. **Roadmap perbaikan sentimen:**
    Tambah sampel negatif, evaluasi metrik yang lebih tepat untuk imbalance (mis.
    ↳ PR-AUC), dan pertimbangkan model berbasis embedding/pretrained yang lebih
    ↳ robust sebelum mengandalkan LSTM untuk deteksi komplain.
"""

display(Markdown(recommendation))

```

1.8.1 Interpretasi 4 — Rekomendasi Bisnis untuk Divisi Fashion & Apparel

Ringkasan Visual (Validasi Kategori Produk): - Model terbaik: **MobileNetV2 Transfer Learning** dengan test accuracy **0.9733** - Baseline: CNN dasar test accuracy **0.9289** vs Transfer Learning **0.9733**

Ringkasan Teks (Analisis Sentimen): Model sentimen utama: **LSTM (Embedding + LSTM)** (threshold tuned **0.56**)

- Accuracy (tuned): **0.9860**
- Negatif — Recall: **0.0385**, F1: **0.0667**, Precision: **0.2500**
- Macro F1: **0.5298**, Balanced Accuracy: **0.5185**, ROC-AUC: **0.1484**

Baseline opsional (lebih stabil untuk triage): **USE + Logistic Regression**

- Accuracy (tuned): **0.9695**
- Negatif — Recall: **0.3077**, F1: **0.2078**, Precision: **0.1569**
- Macro F1: **0.5961**, Balanced Accuracy: **0.6430**, ROC-AUC: **0.8837**

Catatan trade-off target bisnis: - Target recall negatif **0.50** bisa dicapai pada threshold **0.98**, namun precision negatif turun ke **0.0130** (potensi lonjakan beban review manual).

Rekomendasi Eksekutif

1. **Gunakan model visual sebagai validasi kategori otomatis tahap awal.** Terapkan *auto-suggest* kategori ketika confidence tinggi, dan minta konfirmasi ulang jika kategori seller berbeda dari prediksi model.
2. **Siapkan human-in-the-loop untuk kasus ambigu dan kelas yang sering tertukar.** Fokus pada prediksi confidence rendah dan pasangan kategori mirip (contoh Shirts vs Tshirts).
3. **Gunakan sentimen sebagai sistem triage (prioritisasi), bukan keputusan final.** Pada data yang sangat imbalanced, akurasi tinggi tidak menjamin komplain terdeteksi. Operasional sebaiknya memakai target SLA (mis. recall negatif minimal) dan siap menanggung trade-off beban review.
4. **Roadmap perbaikan sentimen:** Tambah sampel negatif, evaluasi metrik yang lebih tepat untuk imbalance (mis. PR-AUC), dan pertimbangkan model berbasis embedding/pretrained yang lebih robust sebelum mengandalkan LSTM untuk deteksi komplain.

1.9 Interpretasi 4 — Strategi Implementasi Model Visual & Roadmap Perbaikan Sentimen

Rekomendasi di bagian ini merangkum hasil eksperimen visual dan teks pada notebook, dengan penekanan pada metrik yang relevan untuk data *imbalanced* (khususnya deteksi ulasan negatif) dan keputusan implementasi yang feasible.

1.9.1 1) Validasi Kategori Produk (Visual) — SIAP IMPLEMENTASI

- Model terbaik pada eksperimen 3-kelas adalah **MobileNetV2 Transfer Learning** dengan Test Accuracy **0.9778** (lebih tinggi daripada CNN dasar **0.9289**).
- **Action items:**
 - Deploy model visual sebagai **auto-suggest kategori** saat seller upload; trigger auto-suggest ketika confidence model ≥ 0.85 .
 - Maintain jalur **manual review** untuk kasus confidence < 0.80 dan untuk pasangan kategori yang sering tertukar (Shirts vs Tshirts).
 - Estimasi beban ulang: Jika 20% upload memerlukan review, cost per review ≈ 2 menit $\rightarrow$ total 5–10 jam reviewer per 10.000 upload.

1.9.2 2) Analisis Sentimen Otomatis (Teks) — BELUM SIAP OPERASIONAL; ROADMAP PERBAIKAN DIPERLUKAN

Masalah Utama Model LSTM: Model LSTM menunjukkan Accuracy tinggi (**0.9860** setelah tuning), **TETAPI ROC-AUC ekstrem rendah (0.1484)** — menandakan model **gagal memisahkan kelas positif dan negatif secara efektif**. Konsekuensi: - Recall negatif **sangat rendah (0.0385)** $\rightarrow$ 96.5% komplain pelanggan terlewat. - Confusion matrix: hanya 1 dari 26 ulasan negatif terdeteksi. - **Model LSTM saat ini TIDAK COCOK untuk deteksi otomatis komplain.**

Solusi Interim: Baseline USE + Logistic Regression Untuk *trriage* komplain saat ini, **gunakan USE + Logistic Regression** (lebih robust untuk data imbalanced): - ROC-AUC: **0.8837** (jauh lebih informatif) - Recall negatif: **0.3077** (masih rendah dari target, tapi lebih baik dari LSTM) - Action items: - Deploy USE+LogReg sebagai sistem prioritisasi awal (bukan keputusan

final). - Flag ulasan prediksi “negatif” untuk human review (triage tool, bukan automation). - Estimasi: Dengan recall 30.77%, 1 dari 3 komplain sebenarnya terdeteksi → tetap perlu human validation.

Roadmap Perbaikan (Jangka Panjang):

1. **Tambah data negatif:** 84 ulasan negatif vs 6.316 positif sangat imbalanced. Target tambah 500–1.000 ulasan negatif berkualitas dari customer complaints atau expert-labeled.
2. **Evaluasi metrik yang tepat untuk imbalance:** Gunakan PR-AUC (Precision-Recall AUC), F1 macro, atau balanced accuracy sebagai utama, bukan accuracy biasa.
3. **Eksplorasi model alternatif:** Pre-trained BERT/RoBERTa + fine-tuning lebih robust daripada LSTM dari nol pada data kecil.
4. **Target SLA:** Tetapkan recall minimum 0.50 untuk deteksi negatif sebelum automate sepenuhnya.

1.9.3 3) Ringkasan Perbandingan Model (Imbalance-Aware)

Metrik	CNN Dasar (Visual)	MobileNetV2 (Visual)	LSTM (Teks)	USE+LogReg (Teks)
Test Accuracy	0.9289	0.9778	0.9860	0.9695
ROC-AUC	—	—	0.1484	0.8837
Recall Negatif	—	—	0.0385 (FAIL)	0.3077 (PARTIAL)
Status	Baseline	Recommended	NOT Ready	Interim Solution

1.9.4 4) Rekomendasi Eksekutif (Ringkas)

1. **Visual:** Deploy MobileNetV2 dengan auto-suggest + human review fallback untuk confidence rendah. **Kesiapan: HIGH (95%)**
2. **Sentimen:** Gunakan USE+LogReg untuk triage (prioritisasi) komplain, bukan keputusan final. LSTM memerlukan perbaikan data/model sebelum operasional. **Kesiapan: LOW (30%) — Diperlukan Roadmap Perbaikan**
3. **Timeline:** Visual bisa go-live 1–2 minggu. Sentimen perlu 4–8 minggu untuk perbaikan dan revalidasi.

Dengan strategi ini, tim operasional mendapat efisiensi otomatisasi visual sambil menjaga human oversight untuk kasus kritikal (komplain pelangan).

1.10 Penutup

Notebook ini menyelesaikan rangkaian eksperimen Deep Learning Modul 8 menggunakan dataset asli Kaggle, dengan fokus pada dua use-case operasional Tokopedia (Fashion & Apparel): validasi kategori produk (visual) dan klasifikasi sentimen ulasan (teks).

Ringkasan hasil kunci: - **Visual (3 kelas, 1.500 gambar subset):** MobileNetV2 Transfer Learning mencapai Test Accuracy **0.9778** (lebih baik dari CNN dasar **0.9289**). Ini menunjukkan transfer learning sangat efektif untuk prototipe dengan data terbatas. - **Teks (10.000 ulasan subset):** LSTM memiliki Accuracy tinggi (**0.9860**), namun deteksi ulasan negatif masih lemah (Recall

negatif **0.0385**). Baseline USE+LogReg memberi sinyal yang lebih baik untuk triage (ROC-AUC **0.8837**, Recall negatif **0.3077**), tetapi masih perlu peningkatan untuk memenuhi target SLA.

Kesimpulan bisnis: modul visual sudah cukup kuat untuk dijadikan kandidat implementasi awal dengan human-in-the-loop. Untuk modul sentimen, fokus berikutnya adalah meningkatkan kemampuan menangkap ulasan negatif (komplain) melalui perbaikan data, metrik evaluasi yang tepat untuk imbalance, dan pemilihan model yang lebih robust.